

A project report on

**AN AI-DRIVEN SAFETY
AND TELEMETRY RISK ANALYTICS
PLATFORM FOR PROACTIVE FLIGHT DATA
MONITORING**

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

with

Specialization in Data Science

by

YASH NAYAN 22BDS0274



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
(SCOPE)**

May,2026

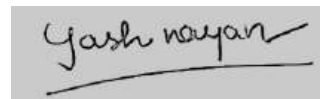
Declaration

I hereby declare that the thesis entitled “**AN AI-DRIVEN SAFETY AND TELEMETRY RISK ANALYTICS PLATFORM FOR PROACTIVE FLIGHT DATA MONITORING**” submitted by me, for the award of the degree of *Bachelor’s of Technology in Computer Science and Engineering with Specialization in Data Science* to VIT is a record of bonafide work carried out by me under the supervision of Vasumathi Elango (Manager), Mahammed Asraf (Tech-Lead) and Palak Mahajan (Mentor).

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: 12th May 2026

A rectangular box containing a handwritten signature in black ink. The signature appears to read "Yash nayyar" with a horizontal line underneath.

Signature of the Candidate

Internship completion certificate

AIRBUS

TO WHOMSOEVER IT MAY CONCERN

This is to confirm that Mr./ Mrs./ Ms. Yash Nayan, is as on 08-May-2026 working with Airbus India Private Limited (formerly known as Airbus Group India Private Limited) as Intern - Full Stack Software Engineer with AI/ML since 04-Feb-2026.

For Airbus India Private Limited (Formerly Airbus Group India Private Limited)



Name: Prema SURESH

Designation: HO I&D & HR Ops, Regional SHRBP ADS

Date: 08-May-2026

Corporate Office:
15th Floor, Block 1, Titanium Tech Park,
ITPL Main Road, Near Kadugodi Tree Park Metro Station,
Kadugodi Sadamangala Industrial Area,
Bengaluru, Bengaluru Urban,
Karnataka- 560067
Phone: +91 (0) 80 66 380 380, Fax: +91 (0) 80 66 380 118

Registered Office & Training Centre:
Airbus India Private Limited
CIN: U74999DL2007PTC457551
Plot no. LP7A-2, Opposite Terminal 3,
Indira Gandhi International Airport, Terminal District,
New Delhi- 110037.
Phone: +91 (0) 11 45 801 100 Fax: +91 (0) 11 45 801
Email: contact.india@airbus.com

Note: The internship is currently ongoing; therefore, the Internship Completion Certificate cannot be provided at this stage by Airbus. Instead, the company has provided the Employment Letter as proof of internship engagement.

ABSTRACT

Commercial aviation safety has historically depended on the retrospective analysis of incidents through static, rule-based monitoring of recorded flight data and the manual review of voluntary incident narratives. While these approaches have fostered one of the safest modes of transport in human history, they are inherently reactive. Traditional systems detect only what they have been explicitly programmed to find, leaving subtle, multi-factor precursor patterns and emerging risk themes invisible until an accumulation of incidents finally triggers regulatory or operator action.

This project presents an integrated, AI-driven safety and telemetry risk analytics platform designed to shift flight data monitoring from a reactive process to a proactive one, ultimately supporting aviation safety analysts at airlines and regulatory bodies. The system is built upon the NASA Aviation Safety Reporting System (ASRS) corpus of 38,655 voluntary incident reports spanning two decades, joined with operational data from the Bureau of Transportation Statistics (BTS), the National Transportation Safety Board (NTSB), and the National Oceanic and Atmospheric Administration (NOAA). To ensure trustworthy predictive performance, the framework is grounded in severe methodological discipline: a strict temporal split is applied to prevent future data leakage, and a rigorous audited whitelist ensures no post-incident variables influence pre-incident predictions.

The proposed platform integrates five interconnected machine learning capabilities. First, a leakage-strict severity triage model combines pre-circumstance tabular metadata with PCA-compressed Sentence-BERT narrative embeddings, applies an Ordinal LightGBM cumulative logit decomposition, calibrates probabilities through temperature scaling, and selects the predicted class through cost-sensitive decision under an asymmetric cost matrix that penalises missed critical incidents one hundred times more heavily than minor over-triage. Second, a three-tier incident category classifier progresses from a TF-IDF baseline through a Sentence-BERT text tower to a Fusion Multi-Layer Perceptron trained with focal loss, per-label positive-class weighting, label smoothing, GELU activations, and BatchNorm under a cosine learning rate schedule. Third, a pre-flight risk scoring methodology combines case-control sampling with prior shift correction to produce deployment-ready calibrated probabilities, with a 200-step threshold sweep tuning the operational decision boundary. Fourth, an unsupervised emerging-risk discovery pipeline clusters sentence embeddings and applies Pruned Exact Linear Time (PELT) changepoint detection to flag structural shifts in risk themes before they escalate. Finally, a contributing-factor knowledge graph applies Louvain community detection and betweenness centrality analysis to identify bridge factors connecting disparate safety risks.

These five capabilities are delivered through an interactive Streamlit analyst dashboard backed by a Python FastAPI inference service exposing the models through a unified GraphQL interface, with trained artifacts loaded into memory at startup. Software reliability is enforced via a Jenkins continuous integration pipeline requiring test passage and strict SonarQube static analysis quality gates at every code change.

Under temporal validation, the severity triage model achieves a Quadratic Weighted Kappa of 0.4612, the category classifier reaches a macro-F1 of 0.7038, and the pre-flight risk model achieves an ROC-AUC of 0.7641 on the held-out test set, demonstrating that an integrated machine learning system grounded in rigorous data hygiene can provide actionable proactive decision support without replacing the judgment of human safety experts.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to my reporting manager **Vasumathi Elango**, my Tech-Lead **Mahammed Asraf** and my team members, **Spoorthy Thammaiah**, **Suraj Prakash**, **Srinath Anandudu**, for their constant guidance, continuous encouragement, and valuable support throughout my internship. Their mentorship, patience, and willingness to share their knowledge greatly contributed to my learning experience and professional growth. My association with them was not limited to academics and work alone, but also provided me with an excellent opportunity to learn from highly skilled professionals and experts in the field of Software Development and DevOps Engineering.

I would like to express my gratitude to our honorable Chancellor **Dr. G. Viswanathan**, Vice-Presidents **Mr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**, and **Dr. G.V. Selvam**, Vice-Chancellor **Dr. V.S. Kanchana Bhaaskaran** and the **Pro-Vice Chancellor Dr. Partha Sharathi Mallick** and our Dean **Dr. Jaisanker N**, School of Computer Science and Engineering (SCOPE) , for providing with an environment to work in and for his inspiration during the tenure of the course.

In jubilant mood I express ingeniously my whole-hearted thanks to <Program char- name>, <Program Chair and designation>, all teaching staff and members working as limbs of our university for their not-self-centered enthusiasm coupled with timely encouragement showered on me with zeal, which prompted the acquirement of the requisite knowledge to finalize my course study successfully. I would like to thank my parents for their support.

It is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this Internship. At last but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly toward the successful completion of this Internship.

Place: Vellore

Date: 12th May 2026

YASH NAYAN

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
	Abstract	i
	Acknowledgement	ii
	List of Tables	v
	List of Figures	vi
	List of Abbreviations	vii
1	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 Motivation	2
	1.3 Scope of the Project	3
	1.3.1 Functional Scope	3
2	PROJECT DESCRIPTION AND GOALS	4
	2.1 Literature Review	4
	2.2 Research Gap	5
	2.3 Objectives	6
	2.4 Problem Statement	6
	2.5 Project Plan	7
3	TECHNICAL SPECIFICATION	9
	3.1 Requirements	9
	3.2 Feasibility Study	10
	3.3 System Specification	12
4	SYSTEM DESIGN	13
	4.1 System Architecture	13
	4.2 Detailed Design	14
	4.3 Machine Learning Model Designs	20
5	METHODOLOGY AND TESTING	23
	5.1 System Development Methodology	23
	5.2 Data Acquisition and Processing Methodology	24
	5.3 Feature Engineering Methodology	26
	5.4 Severity Triage Modelling Methodology (Problem A)	27
	5.5 Category Classification Methodology (Problem B)	28
	5.6 Pre-Flight Risk Modelling Methodology (Problem C)	29

	5.7 Emerging Risk Discovery Methodology (Problem D)	29
	5.8 Contributing Factor Graph Methodology (Problem E)	29
	5.9 Evaluation Metrics Methodology	30
	5.10 Testing Methodology	30
6	PROJECT IMPLEMENTATION	31
	6.1 Development Environment and Tools	31
	6.2 Backend Implementation	31
	6.3 Frontend Implementation	33
	6.4 Deployment Implementation	38
7	RESULTS AND DISCUSSION	40
	7.1 System Functional Results	40
	7.2 Severity Triage Results (Problem A)	40
	7.3 Category Classification Results (Problem B)	41
	7.4 Pre-Flight Risk Results (Problem C)	43
	7.5 Emerging Risk Discovery Results (Problem D)	45
	7.6 Contributing Factor Graph Results (Problem E)	47
	7.7 System Performance Results	49
	7.8 Rag Retrieval and Diversity Discussion	49
	7.9 Discussion	49
8	Conclusion and Future Work	51
9	INTERNSHIP DESCRIPTION	54
	REFERENCES	58
	APPENDIX	59

List of Tables

Table No.	Title	Page No.
7.1	Problem A Severity Classification Test Set Results	40
7.2	Problem B Three-Tier Category Classification Comparison	42
7.3	Problem C Pre-Flight Risk Test Set Results	43
7.4	Problem D Top Emerging Risk Topics	45
7.5	Problem E Highest-Severity Contributing Factor Patterns	47

List of Figures

Figure No.	Title	PageNo.
2.1	Work Breakdown Structure	8
2.2	Gantt Chart	8
4.1	System Architecture Diagram	13
4.2	Entity-Relationship Diagram	15
4.3	Data Flow Diagram (Level 1)	15
4.4	Class Diagram	16
4.5	Use Case Diagram	17
4.6	Sequence Diagram — Severity Prediction	18
4.7	Sequence Diagram — Service Startup and Model Loading	18
4.8	Activity Diagram — End-to-End Severity Prediction	19
4.9	Component Diagram	19
4.10	Deployment Diagram	20
4.11	Three-Tier Category Model Architecture	21
5.1	V-Model Lifecycle Adopted in System Development	24
5.2	Five-Stage Data Pipeline	25
6.1	Streamlit Dashboard Landing Page	33
6.2	Severity Triage Page	34
6.3	Category Classification Page	35
6.4	Pre-Flight Risk Page	36
6.5	Emerging Risk Discovery Page	37
6.6	Contributing Factor Graph Page	38
7.1	Problem A Confusion Matrix	40
7.2	Problem B Tier-by-Tier Macro F1 Comparison	42
7.3	Problem C ROC and Precision-Recall Curves	44
7.4(a)	Problem D Topic Risk Score Distribution	46
7.4(b)	Problem D PELT Changepoint Detection Example	46
7.5	Problem E Louvain Community Visualisation	48
9.1	Team’s Agility Storyboard for Story distrubution	55
9.2	Demo Jenkins CI/CD Pipeline	56
9.3	Demo SonarQube Quality Gate overview	56

List of Abbreviations

Abbreviation	Full Form
ASGI	Asynchronous Server Gateway Interface
ASRS	Aviation Safety Reporting System
AUC	Area Under the Curve
BERT	Bidirectional Encoder Representations from Transformers
BiLSTM	Bidirectional Long Short-Term Memory
CI/CD	Continuous Integration / Continuous Deployment
DO-178C	RTCA Software Certification Standard (Aviation)
EASA	European Union Aviation Safety Agency
ECE	Expected Calibration Error
FOQA	Flight Operations Quality Assurance
HDBSCAN	Hierarchical Density-Based Spatial Clustering
ICAO	International Civil Aviation Organization
JWT	JSON Web Token
LightGBM	Light Gradient Boosting Machine
LLM	Large Language Model
METAR	Meteorological Aerodrome Report
MLP	Multi-Layer Perceptron
NER	Named Entity Recognition
NOAA	National Oceanic and Atmospheric Administration
NTSB	National Transportation Safety Board
PELT	Pruned Exact Linear Time (change-point detection)
PR-AUC	Precision-Recall Area Under Curve
QWK	Quadratic Weighted Kappa
SAFe	Scaled Agile Framework
SBERT	Sentence-BERT
SHAP	Shapley Additive Explanations
STM	Structural Topic Modeling
TCAS	Traffic Collision Avoidance System
TF-IDF	Term Frequency – Inverse Document Frequency
UMAP	Uniform Manifold Approximation and Projection

Chapter 1

1. INTRODUCTION

1.1 BACKGROUND

Commercial aviation is the safest form of long-distance transport ever engineered by humans. The accident rate for scheduled commercial flights has fallen by more than two orders of magnitude since the 1960s, despite enormous growth in the global fleet, route networks, and passenger volume. This safety record is the product of a deliberate, multi-layered safety culture combining rigorous certification, mandatory incident reporting, structured accident investigations, and continuous post-flight analysis of recorded operational data.

Two practices form the operational backbone of modern airline safety analysis: Flight Operations Quality Assurance (FOQA) and Flight Data Monitoring (FDM). Every commercial flight produces a stream of recorded data describing aircraft behaviour, crew actions, and environmental conditions. Patterns in this data, analysed across thousands of flights, reveal precursors to accidents that no single flight crew or maintenance event could detect alone. Airlines feed this data into FOQA programs to flag operational exceedances, such as unstable approaches or hard landings. Concurrently, voluntary reporting programs like the NASA Aviation Safety Reporting System (ASRS) capture the human factors of operational events through narratives submitted by pilots and controllers, while bodies like the National Transportation Safety Board (NTSB) maintain authoritative accident records.

However, these mature programs share a common limitation: the analysis they enable is mostly retrospective and rule based. FOQA exceedances rely on static thresholds, successfully flagging specific events but missing subtle precursor patterns that combine conditions which are individually within tolerance but jointly indicate elevated risk. Furthermore, new risk themes remain invisible until accumulated incident reports force regulatory action, often years after the underlying risk first appeared. Additionally, FOQA data, voluntary incident narratives, accident records, and weather observations are collected and analysed in disconnected pipelines, severely limiting proactive investigations.

1.1.1 Existing Solutions and Their Limitations

Many commercial and academic systems address parts of the safety analytics problem, but none integrate the full range of capabilities required for proactive analysis.

- **Commercial FOQA Platforms:** Mature and well-validated for structured exceedance monitoring, but they remain almost entirely rule-based and proprietary.
- **Open-Source Tools:** Focus primarily on text mining of narrative reports but rarely integrate structured flight data or pre-flight risk modelling.
- **Generic Large Language Models (LLMs):** Can answer general aviation questions but fail to produce structured analytic outputs, perform strict severity triage, or generate calibrated risk probabilities grounded in real incident data.
- **Data Leakage Vulnerabilities:** A common pitfall in incident data modelling is the unintended inclusion of post-incident outcome fields among the predictors. Models predicting severity from features that already contain documented outcomes produce impressive metrics that instantly collapse when deployed against new flights where outcomes are unknown. Few publicly described systems take an explicit, audited stance on preventing this leakage.

1.1.2 Relevant Technologies and Theoretical Foundations

The system proposed in this project is built upon several well-established theoretical and technological frameworks:

- **Gradient Boosted Decision Trees:** Algorithms like LightGBM and CatBoost handle mixed-type features and high-cardinality categorical fields natively, consistently ranking as top performers on structured prediction tasks while offering essential feature importance analysis.
- **Sentence-BERT (SBERT):** Produces fixed-dimension vector embeddings capturing semantic similarity at the sentence and short-document level, enabling effective classification and clustering of narrative incident reports.
- **Isotonic Regression:** Applied as a post-training calibration layer to transform raw model scores into probabilities matching the empirical positive rate, which is critical for downstream threshold-based decision making.
- **Pruned Exact Linear Time (PELT):** Detects structural changepoints in time-series data, distinguishing transient spikes from fundamental shifts in emerging risk topic frequencies.
- **Louvain Community Detection:** Partitions large graphs into densely connected communities, combining with betweenness centrality to identify clusters of related risk factors and the bridge factors connecting them.

1.2 MOTIVATION

This project is driven by a specific, highly visible gap in the public aviation safety data ecosystem, focusing on three main aspects:

- **Problem Significance:** The raw material for proactive aviation safety analytics is publicly available at an unprecedented scale, including tens of thousands of NASA ASRS narrative reports, Bureau of Transportation Statistics (BTS) on-time performance data, NTSB accident databases, and National Oceanic and Atmospheric Administration (NOAA) weather observations. What is missing is an integrated, methodologically rigorous platform that joins these sources and surfaces actionable patterns to analysts.
- **Practical and Academic Motivation:** Safety analysts currently rely on a patchwork of vendor tools and custom scripts, slowing investigation and fragmenting knowledge. Integrating advanced machine learning techniques into a single platform under shared methodological discipline represents a vital academic and practical step for safety-critical domains.
- **Technical Interest and Innovation:** The project addresses unresolved technical challenges by implementing a strict whitelist-based leakage audit, a cost-sensitive ordinal prediction model penalizing fatal misclassifications 100 times more heavily than minor over-triage, a case-control sampling design with prior shift correction for rare events, and severity-weighted topic risk scoring.

1.3 SCOPE OF THE PROJECT

The proposed AI-driven safety and telemetry risk analytics platform functions as a comprehensive decision support tool for aviation safety analysts at airlines and regulatory bodies.

1.3.1 Functional Scope

The system accepts structured incident metadata, narrative text, and flight-level operational data to produce five classes of analytic output:

1. **Severity Triage:** Estimates likely severity on a four-level ordinal scale using only pre-circumstance metadata, returning calibrated probabilities and a cost-sensitive predicted class minimizing expected misclassification cost.
2. **Incident Category Classification:** Assigns safety categories to reports using a three-tier architecture (TF-IDF, SBERT text tower, and a Fusion Multi-Layer Perceptron).
3. **Pre-Flight Risk Scoring:** Produces a calibrated probability of an incident occurring on a planned flight based on route, time, aircraft, carrier, and forecast weather, handling rare-event base rates via case-control sampling and prior shift correction.
4. **Emerging Risk Discovery:** Identifies emerging risk topics from recent narrative reports by clustering narratives, tracking prevalence, computing severity-weighted risk scores, and flagging structural change points.
5. **Contributing Factor Graph Analysis:** Constructs a knowledge graph of contributing factors, flight phases, aircraft types, and FAR parts, using Louvain community detection and betweenness centrality to surface risk clusters and bridge factors.

1.3.2 Non-Functional Scope

The system supports sub-second response times for inference operations and multi-second response times for graph and emerging risk panels. It features a modular architecture where individual models can be retrained independently. The platform is built for portable deployment, with the entire data and model pipeline reproducible from a fixed global seed. Continuous integration (CI) enforces test passage and static analysis quality gates before any code changes are accepted.

1.3.3 Technical Scope and Project Boundaries

The frontend utilizes a Streamlit-based interactive dashboard backed by a Python FastAPI inference service via a GraphQL interface. Continuous integration is managed through Jenkins, with code quality enforced by SonarQube. Machine learning components are implemented using LightGBM, CatBoost, scikit-learn, PyTorch, Sentence-Transformers, BERTopic, ruptures, and NetworkX. Models are trained offline using 38,655 NASA ASRS voluntary incident reports joined with BTS, NTSB, and NOAA data.

Limitations: The platform processes structured records and narrative text but does not consume real-time avionics data, live flight telemetry, images, or audio. It serves exclusively as a decision support tool, not a flight-critical system; it does not interface directly with aircraft systems, produce certified outputs (e.g., DO-178C or ARP4754A), or replace human judgment. Outputs are designed as starting points for human investigation rather than final automated decisions.

2. PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

The application of computational analytics to aviation safety has evolved significantly over the past two decades, transitioning from static, rule-based systems to advanced deep learning and natural language processing (NLP) architectures. The literature and patent landscape relevant to the proposed platform can be categorized into four primary domains: pre-flight risk assessment, incident severity classification, multi-label narrative categorization, and unsupervised emerging-risk discovery.

2.1.1 Pre-Flight Risk Assessment and Telemetry Analytics

Early approaches to flight risk quantification relied heavily on deterministic rulesets and static database cross-referencing. Adkins established a foundational precedent by patenting a pre-flight risk measuring system that combined static accident-history databases with non-static weather and traffic databases to trigger alerts on a flight-deck computer. While this established the legal and operational framework for pre-departure scoring derived from heterogeneous databases, it lacked the ability to learn complex, non-linear precursor patterns.

More recently, commercial entities have explored machine learning for flight-level risk scoring, primarily for operational and financial optimization. Swiss Reinsurance Company Ltd. patented a system predicting flight trajectory deviations and delay risks, utilizing historical phase tables to automate risk-transfer pipelines. This confirmed the commercial viability of ML-driven flight scoring but focused strictly on delay rather than safety severity. Furthermore, Honeywell International Inc. introduced a qualitative risk assessment method for aircraft, rendering risk values derived from equipment state, crew factors, and environmental context directly onto avionic displays. However, these systems fundamentally rely on qualitative or rule-driven heuristic scoring rather than calibrated, data-driven probabilities derived from case-control matched incident corpora.

2.1.2 Aviation Incident Severity and Risk Classification

The challenge of classifying the severity of an aviation incident using post-flight reports has garnered significant academic interest. Zhang and Mahadevan proposed an ensemble machine learning model for aviation incident risk prediction, employing a hybrid Support Vector Machine (SVM) and Deep Neural Network (DNN) architecture. Their work demonstrated the efficacy of fusing narrative text with contextual, structured features to classify reports into five distinct risk levels.

Building upon this, Wang et al. developed a fusion model combining Convolutional Neural Networks (CNNs) for structured data and Bidirectional Long Short-Term Memory (BiLSTM) networks for narrative text. This architecture successfully integrated multiple modalities to identify risk levels. However, a persistent limitation in these studies is the implicit vulnerability to data leakage. The feature sets often incorporate post-incident outcomes (e.g., component failures, damage assessments) to predict the severity of the event, generating artificially high-performance metrics that do not reflect true predictive capability against new, uncompleted flights.

2.1.3 Narrative Classification and Natural Language Processing

The NASA Aviation Safety Reporting System (ASRS) contains a rich corpus of unstructured narrative text, prompting extensive research into automated categorization. Early approaches utilized weakly supervised learning. Zhao and Ng applied a semantic lexicon framework to identify causal factors in ASRS reports, treating the task as a multi-label text classification problem. This validated that ASRS narratives contain sufficient signal for causal factor extraction, though it relied heavily on rigid lexical constraints.

The advent of transformer-based architectures revolutionized this domain. Kierszbaum and Lapasset applied a distilled BERT model for question-answering on ASRS reports, demonstrating that transformer language models could successfully extract structured semantic answers from domain-specific free text. Chandra et al. significantly advanced this by developing "Aviation-BERT," a domain-adaptive pre-trained model fine-tuned on over 540,000 NTSB and ASRS narratives. Their work proved that domain-specific pre-training substantially outperforms generic models (like BERT-base-uncased) on downstream aviation classification tasks.

2.1.4 Topic Modeling and Emerging Risk Discovery

Unsupervised discovery of latent safety themes is critical for proactive hazard mitigation. Kuhn applied Structural Topic Modeling (STM) to ASRS near-miss reports, proving that probabilistic topic modeling could surface latent themes and temporal trends without predefined taxonomies.

Recent advancements have transitioned from STM to embedding-based clustering. Mumtaz et al. proposed a domain-adapted BERTopic framework (SafeCivAirBERT) to mine and track the temporal evolution of civil aircraft operational risk factors. Their research demonstrated that coupling domain-adaptive pre-training with BERTopic yields highly coherent, stable, and interpretable topics. Nevertheless, prior art largely treats topic modeling as an isolated analytical exercise, rarely integrating it with automated, real-time changepoint detection or severity-weighted scoring systems designed for immediate analyst triage.

2.2 RESEARCH GAP

While the literature and patent landscape demonstrate substantial progress in isolated aviation machine learning tasks, a comprehensive review reveals four critical research and methodological gaps that the proposed platform addresses:

1. **Methodological Indiscipline and Data Leakage:** The most severe gap in existing severity prediction literature is the lack of a strict, audited boundary between pre-circumstance predictors and post-incident outcomes. Models frequently learn to predict severity using features that describe the incident's aftermath (e.g., "aircraft destroyed" or "emergency declared"). There is a profound need for a system built upon a strict feature whitelist that evaluates true predictive capacity using solely pre-incident metadata.
2. **Absence of Calibration and Cost-Sensitive Evaluation:** Existing systems largely evaluate classification performance using standard accuracy or F1 scores. In aviation safety, missing a critical incident is orders of magnitude more dangerous than over-triaging a minor one. Current research fails to implement asymmetric cost matrices and rarely subjects raw model logits to isotonic or Platt calibration, which is mandatory if predictions are to be used for operational threshold-based alerting.
3. **Positive-Unlabeled Identifiability in Pre-Flight Risk:** Models predicting the likelihood of an incident on a future flight often fail to account for the extreme class imbalance (the

rare-event base rate). Systems trained exclusively on ASRS data lack the non-incident denominator required to calculate true risk probabilities. A methodological gap exists for an approach utilizing case-control sampling matched with operational non-incident data, combined with mathematically rigorous prior-shift correction.

4. **Architectural Fragmentation:** The prior art presents severity classifiers, NLP categorizers, and topic models as disconnected, standalone scripts or academic exercises. There is no publicly available, reproducible platform that integrates these disparate capabilities—alongside graph-based causal analysis—into a unified inference service with strict temporal data splitting and explicit model provenance for end-user analysts.

2.3 OBJECTIVES

The primary aim of this project is to deliver an integrated, reproducible aviation-safety analytics platform that addresses the identified research gaps through a shared data pipeline, a unified inference service, and a single analyst dashboard. To achieve this aim, the following specific objectives have been established:

- To curate and integrate disparate public aviation datasets—including NASA ASRS, BTS on-time performance, NTSB accident records, and NOAA METAR weather data—into a single reproducible, temporally split (train/validation/test) corpus.
- To design a deterministic severity rubric and a multi-label category taxonomy that engineer supervised learning targets from structured ASRS fields without relying on subjective human re-labelling.
- To implement and enforce a strict, version-controlled feature whitelist that prevents post-incident outcome fields from leaking into predictive models, validated automatically at every continuous integration build.
- To develop, calibrate, and evaluate five distinct machine learning pipelines: a cost-sensitive gradient-boosted severity classifier, a three-tier multi-modal category classifier, a prior-shift corrected pre-flight risk scorer, a BERTopic-based emerging-risk discovery pipeline, and a contributing-factor knowledge graph.
- To architect a unified inference backend using FastAPI and GraphQL, serving all five analytic capabilities with explicit model-status provenance (e.g., TRAINED, STUB, UNAVAILABLE) attached to every prediction.
- To design and deploy an interactive, front-end dashboard that allows safety analysts to explore live predictions, conduct "what-if" explorations, and visualize complex networks and temporal changepoints.
- To package the entire platform as containerized microservices, supported by a robust Continuous Integration (CI) pipeline that enforces test passage, code-style consistency, and static-analysis quality gates on every code commit.

2.4 PROBLEM STATEMENT

Despite the abundance of high-quality aviation safety data—including the NASA Aviation Safety Reporting System (ASRS), NTSB accident records, BTS flight data, and NOAA weather telemetry—current analytic capabilities remain highly fragmented and reactive. Specifically:

- Severity triage of incoming incident reports is largely a manual, labor-intensive process.
- Incident categorization relies on subjective human assessment against internal taxonomies.
- Pre-flight risk reasoning is typically operator-specific and rule-based rather than driven by calibrated probabilities.

- The discovery of emerging risk themes is reactive, typically occurring only after post-hoc regulatory action.
- Contributing-factor analysis is relegated to static spreadsheets, obscuring complex systemic failures.

The Exact Problem: There is an absence of a single, methodologically disciplined machine learning platform that integrates these fragmented capabilities while strictly enforcing data hygiene—specifically leakage prevention, temporal validation, and probability calibration. To solve this, the project decomposes the analytics gap into five specific sub-problems:

1. **Severity Triage:** Predicting incident severity using only pre-circumstance metadata to prevent data leakage.
2. **Category Classification:** Assigning multi-label safety categories by fusing structured metadata with free-text narratives.
3. **Pre-Flight Risk Scoring:** Estimating the calibrated probability of an incident for a planned flight, utilizing case-control sampling and prior-shift correction to handle rare-event base rates.
4. **Emerging-Risk Discovery:** Identifying shifting narrative themes using unsupervised clustering and changepoint detection.
5. **Contributing-Factor Graph:** Constructing a knowledge graph to identify highly severe, recurring edge patterns and systemic bridge factors.

2.5 PROJECT PLAN

The project follows a hybrid SAFe-style iterative methodology layered over a strict V-model verification discipline. The development lifecycle is structured into seven sequential but overlapping phases:

- **Phase 1 — Requirement Analysis and Planning:** Identifying project scope and objectives, surveying public aviation-safety data sources, defining the five analytic sub-problems, and selecting the technology stack.
- **Phase 2 — Data Engineering and Pipeline Construction:** Implementing the automated ingestion pipeline for NASA ASRS, BTS, NTSB, and NOAA datasets. This includes target engineering, executing the strict feature whitelist to prevent leakage, and enforcing the temporal dataset split (train, validation, and test sets).
- **Phase 3 — Model Development:** Developing and calibrating the core machine learning components, including the CatBoost severity classifier, the three-tier category architecture, the pre-flight risk scorer, the BERTopic emerging-risk pipeline, and the heterogeneous contributing-factor graph.
- **Phase 4 — Backend Development:** Building the unified FastAPI inference service, which exposes the five capabilities through a single GraphQL contract. This includes a singleton model store that loads trained artifacts into memory at startup to eliminate deserialization latency.
- **Phase 5 — Frontend Development:** Implementing the analyst-facing Streamlit dashboard. This involves constructing five distinct problem panels that support live predictions, what-if explorations, and interactive network and trend visualizations.
- **Phase 6 — Integration and Testing:** Executing end-to-end integration of the offline pipeline, inference service, and dashboard. This phase enforces continuous integration quality gates (style, complexity, security) and ensures complete reproducibility through identical model artifacts on repeated runs.
- **Phase 7 — Deployment and Documentation:** Packaging the FastAPI service and dashboard as separate Docker containers. This includes configuring the Jenkins CI/CD pipeline to

automate testing and registry pushes on every main-branch commit, alongside finalizing the technical report and documentation.

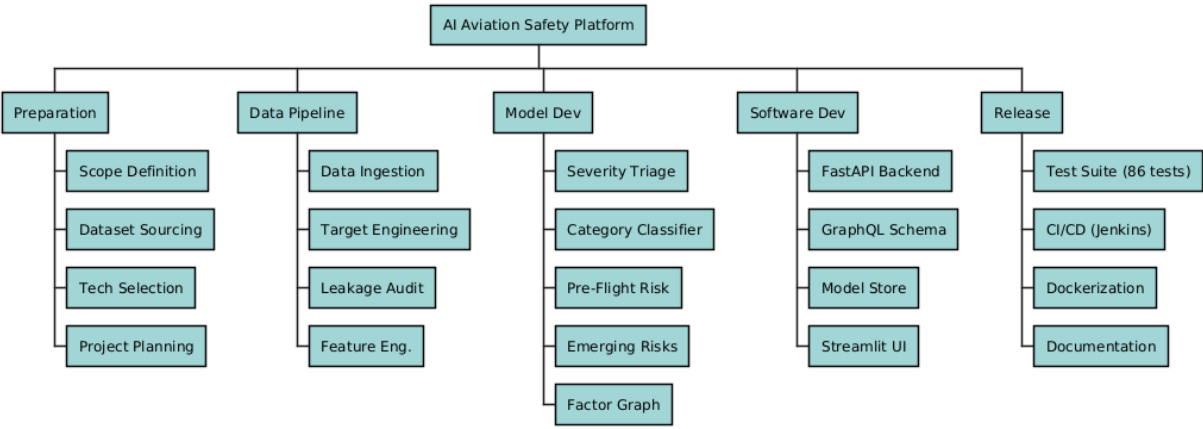


FIGURE 2.1 — Work Breakdown Structure

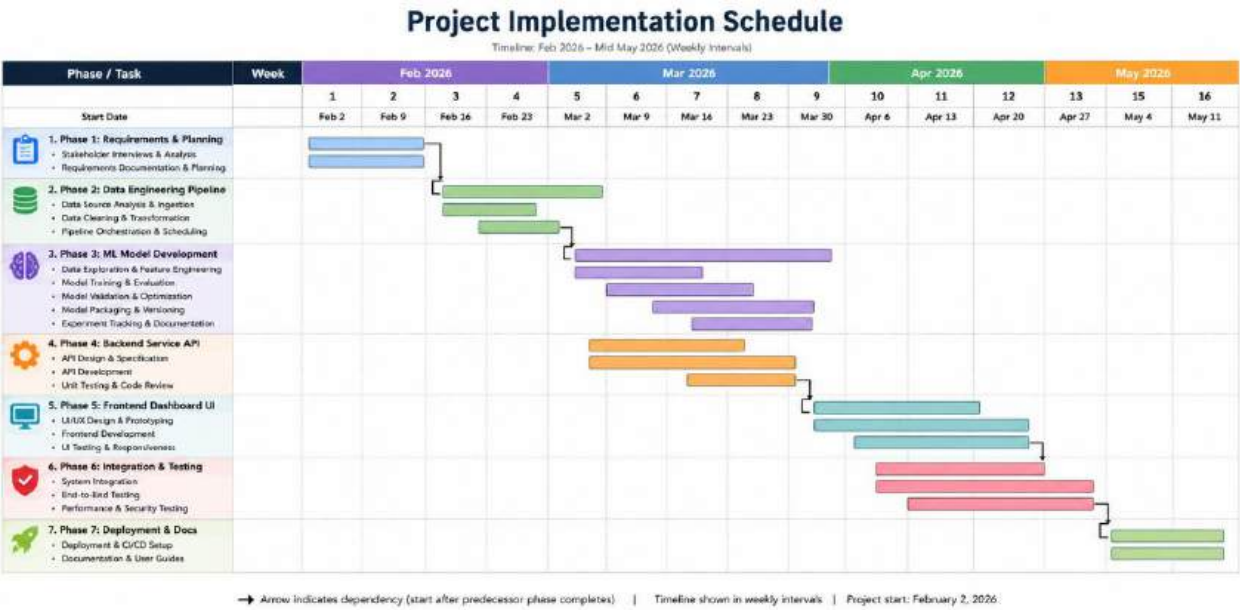


FIGURE 2.2 — Gantt Chart

3. TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

This section captures the functional and non-functional requirements that the platform was built to satisfy. The functional requirements describe what the system does. The non-functional requirements describe the operational qualities such as performance, scalability, security, reliability, and portability that the system was designed to meet.

3.1.1 Functional Requirements

- The system must accept aviation incident metadata, narrative text, and flight-level operational data as input to produce analytic outputs across five interconnected risk problems.
- The system must classify incident severity on a four-level ordinal scale (minor, moderate, substantial, and critical) using only pre-circumstance metadata, ensuring all post-incident outcome fields are excluded via an audited feature whitelist.
- Severity prediction must return calibrated probabilities for each level, utilizing a cost-sensitive predicted class chosen to minimize expected misclassification cost under a configured asymmetric cost matrix.
- The system must classify incident narratives into one or more of five safety categories (flight operations, equipment and systems, ATC and communication, environment, and airspace and navigation) using a three-tier architecture.
- This category classification must progress from a TF-IDF baseline through a Sentence-BERT text tower to a Fusion Multi-Layer Perceptron, supporting per-label decision threshold tuning on a validation set.
- The system must provide a pre-flight risk scoring methodology joining flight-level on-time performance data with weather observations and historical incident records.
- Pre-flight risk scoring must support case-control sampling to handle rare-event base rates, utilizing prior-shift correction and isotonic regression to recover deployment-ready probability estimates.
- The system must perform unsupervised emerging-risk discovery on narrative corpora by clustering texts into topics, tracking topic prevalence over time, and computing a severity-weighted risk score.
- Emerging-risk discovery must detect structural changepoints in topic time series utilizing the Pruned Exact Linear Time (PELT) algorithm and rank top risks for direct analyst review.
- The system must construct a contributing-factor knowledge graph linking contributing factors, flight phases, aircraft type families, and Federal Aviation Regulations (FAR) parts, with edges weighted by co-occurrence frequency.
- The knowledge graph must apply Louvain community detection to identify risk clusters and betweenness centrality to identify bridge factors connecting otherwise separate clusters.
- The system must expose all five capabilities through a unified backend GraphQL interface, powering a multi-panel interactive dashboard that supports visual explorations like confusion matrices, trend overlays, and interactive network graphs.

- The system must guarantee full reproducibility, ensuring every model artifact and processed dataset can be regenerated from raw data through deterministic scripts utilizing a fixed global seed.

3.1.2 Non-Functional Requirements

- **Performance:** The platform must support analyst-level interaction with sub-second response times for severity, category, and pre-flight risk inference, and multi-second response times for larger graph and emerging-risk payloads. Model artifacts must load into memory once at service startup to eliminate prediction latency.
- **Scalability:** The modular architecture must allow individual models to be replaced or retrained independently without disrupting other endpoints. Heavy offline processing (e.g., topic modeling and embedding generation) must run separately from the live inference service.
- **Security:** Backend endpoints must be protected against malformed inputs via GraphQL schema validation, and Cross-Origin Resource Sharing (CORS) must be configured to strictly allow only the dashboard origin.
- **Reliability:** The data pipeline must fail loudly during offline processing if leakage audits, target engineering rules, or temporal split constraints are violated. The system must degrade gracefully (e.g., falling back to Principal Component Analysis and MiniBatchKMeans if primary clustering dependencies fail to load) without breaking downstream logic.
- **Portability:** The system must be fully containerized for deployment on standard Linux environments, while also running seamlessly on macOS and Windows development machines, utilizing YAML files for all human-readable configurations.
- **Maintainability:** Continuous integration (CI) must enforce that every code change passes an extensive test suite and static-analysis quality gates (covering code style, complexity, security, and coverage) before merging.

3.2 FEASIBILITY STUDY

3.2.1 Technical Feasibility

The platform is technically feasible because it is built almost entirely on mature, widely supported, open source technologies. The Python ecosystem provides production grade libraries for every stage of the pipeline. LightGBM and CatBoost are battle tested gradient boosted decision tree implementations. Scikit-learn provides the isotonic calibration, baseline classifiers, and evaluation metrics. Sentence-Transformers provides pre trained sentence embedding models that run on CPU, so no specialised AI hardware is needed. PyTorch supports the Fusion MLP model with standard CPU based training that completes in minutes on a development machine. BERTopic, with its underlying pluggable architecture for dimensionality reduction and clustering, supports the fall back to PCA and MiniBatchKMeans on platforms where the Numba based dependencies are unavailable. The ruptures library implements the PELT changepoint detection algorithm in pure Python with NumPy. NetworkX provides the graph data structures, Louvain community detection, and centrality computations.

FastAPI with the Strawberry GraphQL library provides a modern, asynchronous backend framework with automatic schema validation and interactive query exploration. Streamlit provides a rapid UI development framework that is well suited for analyst facing data exploration interfaces. Jenkins and SonarQube together provide continuous integration and code quality enforcement using technologies that are standard in industrial software development.

The required compute is modest. Severity and category model training run on a single CPU machine within minutes. Sentence-BERT embedding generation for the full ASRS corpus takes a single digit number of minutes on CPU. Topic modelling and changepoint detection are similarly inexpensive. No GPU is required at any stage of the pipeline, although training on a GPU would reduce embedding generation time.

The data is publicly available. The NASA ASRS corpus is hosted on the HuggingFace dataset hub and downloads automatically on first run. The BTS, NTSB, and NOAA datasets are freely available through their respective public portals.

3.2.2 Economic Feasibility

The platform is cost effective because all of its components are free and open source. There are no licensing fees for any of the libraries, frameworks, or development tools used. The datasets are publicly available at no cost. Development can be performed on standard laptop hardware. Production deployment on a single virtual machine with eight gigabytes of memory is sufficient for analyst level use. Cloud deployment is supported by the containerised architecture and can use any of the major cloud providers on a pay as you use basis. Continuous integration through Jenkins can run on a self hosted server or on a managed cloud build service. SonarQube has both a free community edition and paid enterprise editions, and the community edition is sufficient for the static analysis requirements of the project.

The economic risk is low. There are no recurring API costs for inference because all model inference happens locally within the FastAPI service. There are no proprietary data subscriptions. The largest economic consideration is the analyst time required to interpret the system's outputs, which is unaffected by the platform itself and is a constant cost of any safety analysis activity.

3.2.3 Operational Feasibility

Operationally, the platform fits naturally into the existing workflow of an aviation safety analyst. The dashboard provides a familiar interactive interface with separate panels for each analytic capability. Severity triage produces a single ordinal class along with calibrated probabilities, which is the same output format that analysts already work with in vendor FOQA tools. Category classification produces a small set of multi label assignments that map directly to existing safety categorisation taxonomies. Preflight risk scoring produces a single calibrated probability that can be threshold compared in the same way as existing risk indicators. Emerging risk discovery and factor graph analysis surface new patterns for human review, with the explicit framing that the system suggests starting points for investigation rather than producing final automated decisions. The operational risk is also bounded by design. The platform is a decision support tool, not a flight critical system. It does not interface with aircraft, does not influence in flight decisions, and does not produce certified outputs. Its failure modes are limited to producing incorrect or low quality recommendations to analysts, who remain in the loop for every consequential decision. The strict leakage audit, the cost sensitive prediction with one hundred fold penalisation of fatal misclassifications, the temporal validation that prevents future data leakage, and the calibration that produces deployment ready probabilities together constitute a methodological discipline that makes the system's outputs trustworthy enough to serve as inputs to human investigation.

Adoption is straightforward. The dashboard requires no specialised training beyond familiarity with aviation safety concepts. The underlying methodology is documented and reproducible. The test suite of eighty six tests provides ongoing assurance that pipeline changes do not silently

degrade behaviour. The system can be deployed on premises for organisations that prefer to keep safety data in house, or in a private cloud environment for organisations with cloud first policies.

3.3 SYSTEM SPECIFICATION

3.3.1 Hardware Specification

The hardware requirements for both development and deployment are modest, as the platform is designed to operate on standard CPU hardware.

- **Processor:** Intel Core i5 or equivalent, four cores minimum.
- **RAM:** 8 GB minimum for inference-only deployment; 16 GB recommended for full offline pipeline execution (loading datasets, computing SBERT embeddings, fitting BERTopic).
- **Storage:** 10 GB free disk space for raw data, processed artifacts, and trained models.
- **Network:** Stable broadband connection for initial dataset downloads.
- **GPU:** Optional; CPU inference is the default and is sufficiently fast for analyst-level interactive use.
- **Operating System:** Ubuntu 22.04 or later, Windows 10 or later, macOS 12 or later.

3.3.2 Software Specification

The software stack is pinned to specific tested versions to ensure deterministic results across environments.

- **Frontend:** Streamlit (Multi-page interactive dashboard).
- **Backend Framework:** Python FastAPI (Asynchronous HTTP framework).
- **API Interface:** Strawberry GraphQL (Schema-first API).
- **ASGI Server:** Uvicorn (Production-grade server).
- **Tabular ML:** LightGBM (v4.0+) and CatBoost (v1.2+) for gradient-boosted trees.
- **Hyperparameter Tuning:** Optuna (v3.0+) for Bayesian optimization.
- **Deep Learning:** PyTorch (v2.0+) for the Fusion MLP architecture.
- **Embeddings & NLP:** Sentence-Transformers (all-MiniLM-L6-v2 model) and BERTopic (v0.15+).
- **Data Science Utilities:** scikit-learn (v1.3+) for isotonic regression, ruptures (v1.1+) for PELT changepoint detection, and NetworkX (v3.0+) for graph algorithms.
- **Data Handling:** HuggingFace Datasets (v2.14+) and Apache Parquet format.
- **Infrastructure & CI/CD:** Docker (containerization), Jenkins (build automation), SonarQube (static analysis), and PyYAML (configuration management).
- **Language:** Python 3.10 or later.

4. SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The platform utilizes a layered and modular architecture divided into five logical layers: the presentation layer, the API gateway layer, the model inference layer, the offline training and data pipeline layer, and the data and artifact storage layer. A separate continuous integration (CI) and quality enforcement pipeline cuts across all layers. This design keeps each component focused on a single responsibility, enables isolated testing, and allows the offline training pipeline to evolve independently of the live inference service.

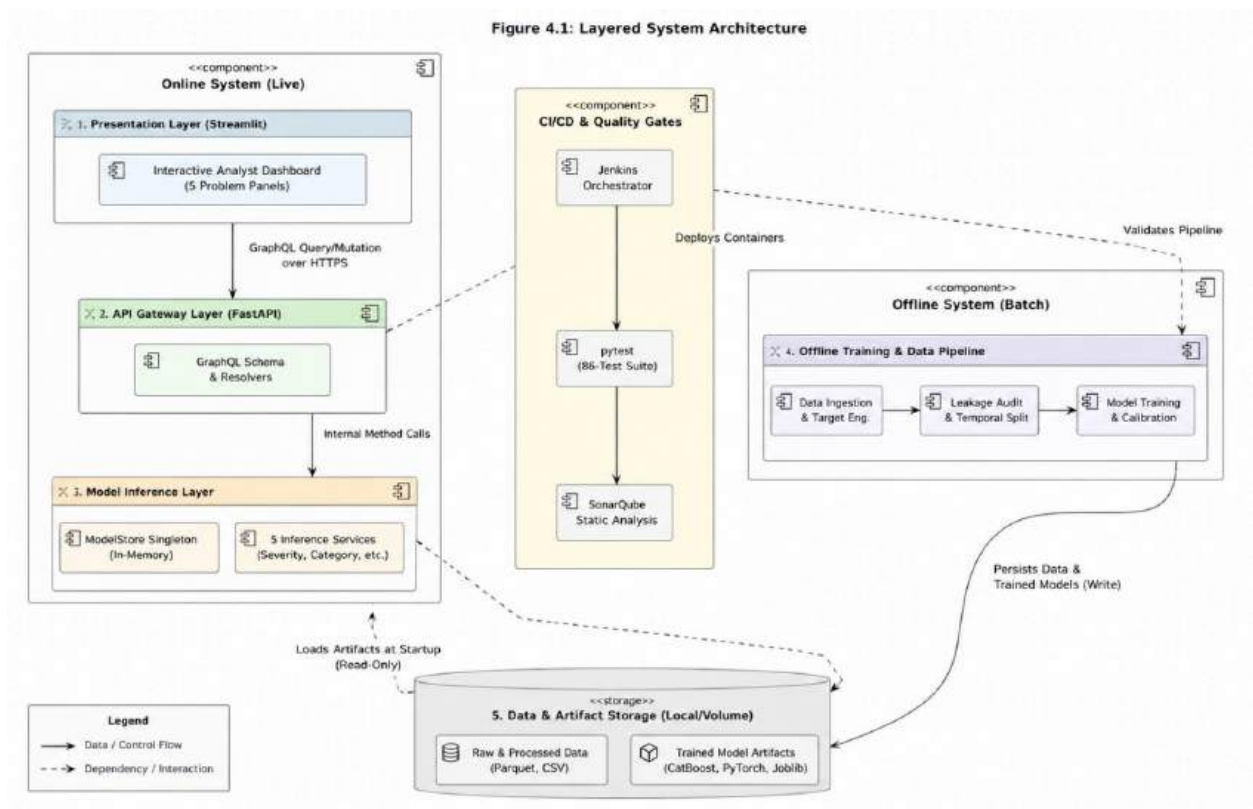


FIGURE 4.1 — System Architecture Diagram

Description: Layered stack showing Presentation, API, Inference, Offline Pipeline, and Storage layers with arrows indicating data and control flow, alongside the cross-cutting CI/CD pipeline.

4.1.1 Presentation Layer

Built using Streamlit, this layer provides a multi-page interactive dashboard tailored for analyst-facing data exploration. It includes an architecture overview landing page and distinct panels for each analytic problem, allowing analysts to transition between capabilities without context switching. The dashboard communicates with the backend exclusively over a GraphQL interface.

4.1.2 API Gateway Layer

Implemented using Python FastAPI and the Strawberry GraphQL library, a single GraphQL endpoint exposes the entire platform's query and mutation surface. GraphQL was selected over REST because it allows the dashboard to fetch the exact combination of model metadata, statistics, and prediction results required for a given screen in a single request. Cross-Origin Resource Sharing (CORS) is strictly configured to allow only the dashboard origin, and a lifecycle hook loads all model artifacts into memory at service startup.

4.1.3 Model Inference Layer

This layer holds the trained model artifacts and prediction logic for each of the five problems. Inference services share a singleton model store that loads artifacts once at startup and holds them in memory for the lifetime of the process. Depending on the request, this layer applies the trained models, executes post-processing (e.g., isotonic calibration, cost-sensitive class assignment), or reads precomputed analytic outputs from the offline pipeline.

4.1.4 Offline Training and Data Pipeline Layer

Operating independently of the inference service, this layer transforms raw input datasets into trained model artifacts and precomputed analytic outputs. The pipeline is structured as a sequence of deterministic phase scripts focusing on data foundation, severity model training, category model training, emerging risk discovery, and factor graph construction.

4.1.5 Data and Artifact Storage Layer

This filesystem-based layer holds three categories of files: raw input data (Apache Parquet format), processed analytic outputs (Parquet, JSON, CSV), and trained model artifacts (CatBoost binaries, LightGBM text files, joblib bundles, and serialized BERTopic models). Each artifact is accompanied by a JSON metadata file containing versioning, training timestamps, and primary metric values.

4.1.6 Continuous Integration and Quality Enforcement

A CI pipeline, managed via Jenkins and SonarQube, runs across all layers. Every code change triggers a build that executes an 86-test suite and static analysis quality gates (covering code style, complexity, security, and coverage). This ensures methodological invariants, such as leakage prevention, are enforced at every commit.

4.2 DETAILED DESIGN

4.2.1 Data Storage and Artifact Schema

The entity-relationship model ties raw data, derived targets, features, model artifacts, and analytic outputs together.

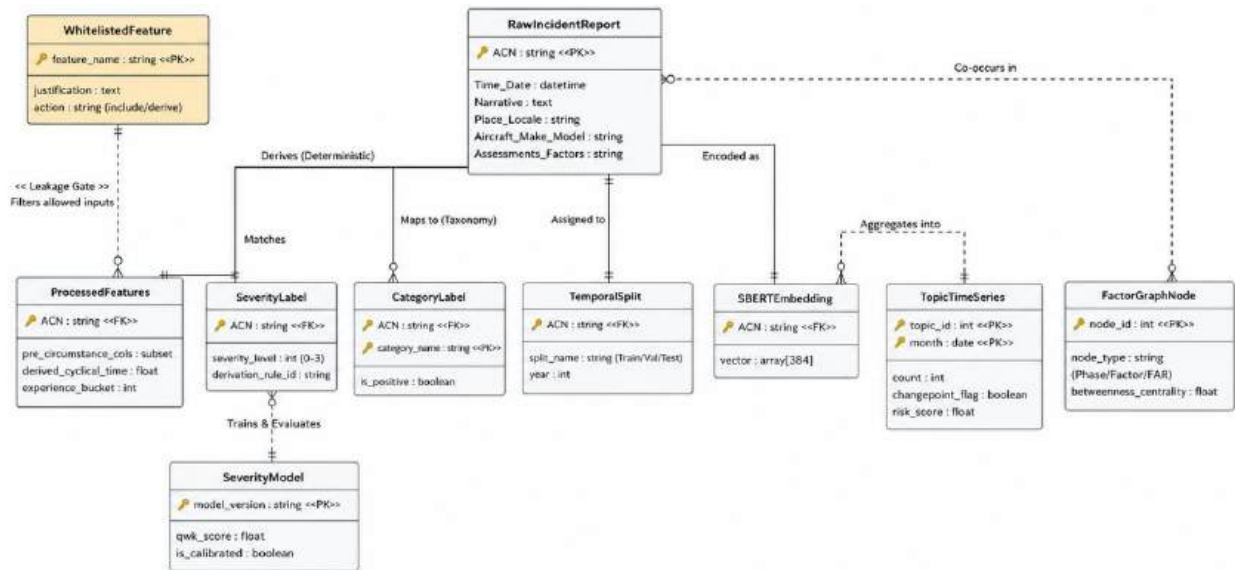


FIGURE 4.2 — Entity-Relationship Diagram

Description: Logical data model showing entities (RawIncidentReport, WhitelistedFeature, ProcessedFeatures, etc.) and their relationships, highlighting how leakage prevention is structurally enforced.

The raw incident report is the central entity, identified by the Accession Control Number (ACN). The most critical constraint in the data model is the whitelist relationship: the processed features entity can only contain columns explicitly defined in the whitelisted feature catalog with an "include" or "derive" action, making leakage prevention a structural property of the system.

4.2.2 Data Flow Design

The end-to-end data flow illustrates how data progresses through the offline pipeline, persists to storage, and is read by the inference service at startup.

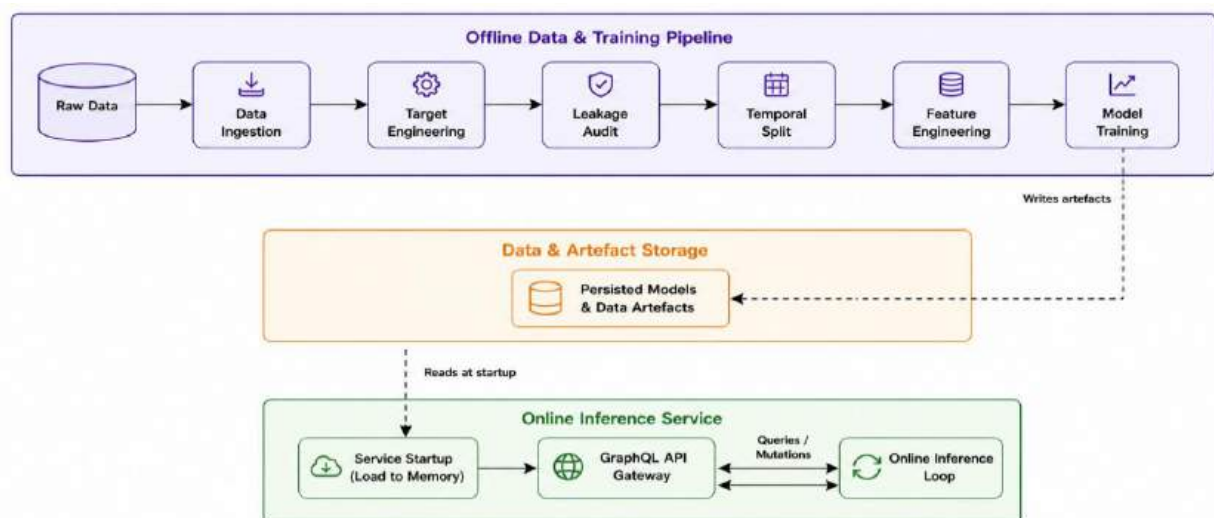


FIGURE 4.3 — Data Flow Diagram (Level 1)

Description: Left to right flow showing offline data ingestion, target engineering, leakage audit, temporal split, feature engineering, and model training, flowing into the online GraphQL inference loop.

4.2.3 Class Diagram

The principal classes in the backend inference layer center around the singleton model store, per-problem service modules, and GraphQL resolvers.

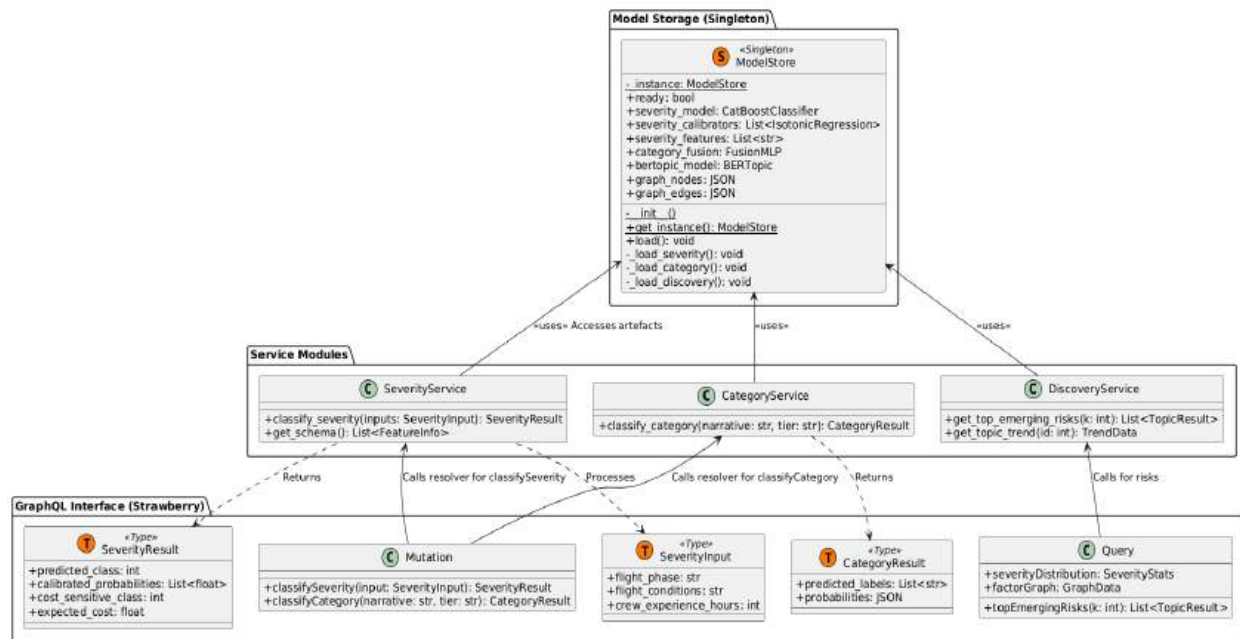


FIGURE 4.4 — Class Diagram

Description: UML class diagram showing the ModelStore singleton, the five Service Modules, and the GraphQL Query/Mutation layer.

4.2.4 Use Case Diagram

Two primary actors (Aviation Safety Analyst, Safety Manager) interact with the frontend dashboard, while three secondary actors (Data Engineer, ML Engineer, CI System) interact with the offline pipeline and infrastructure.

Figure 4.5: Use Case Diagram

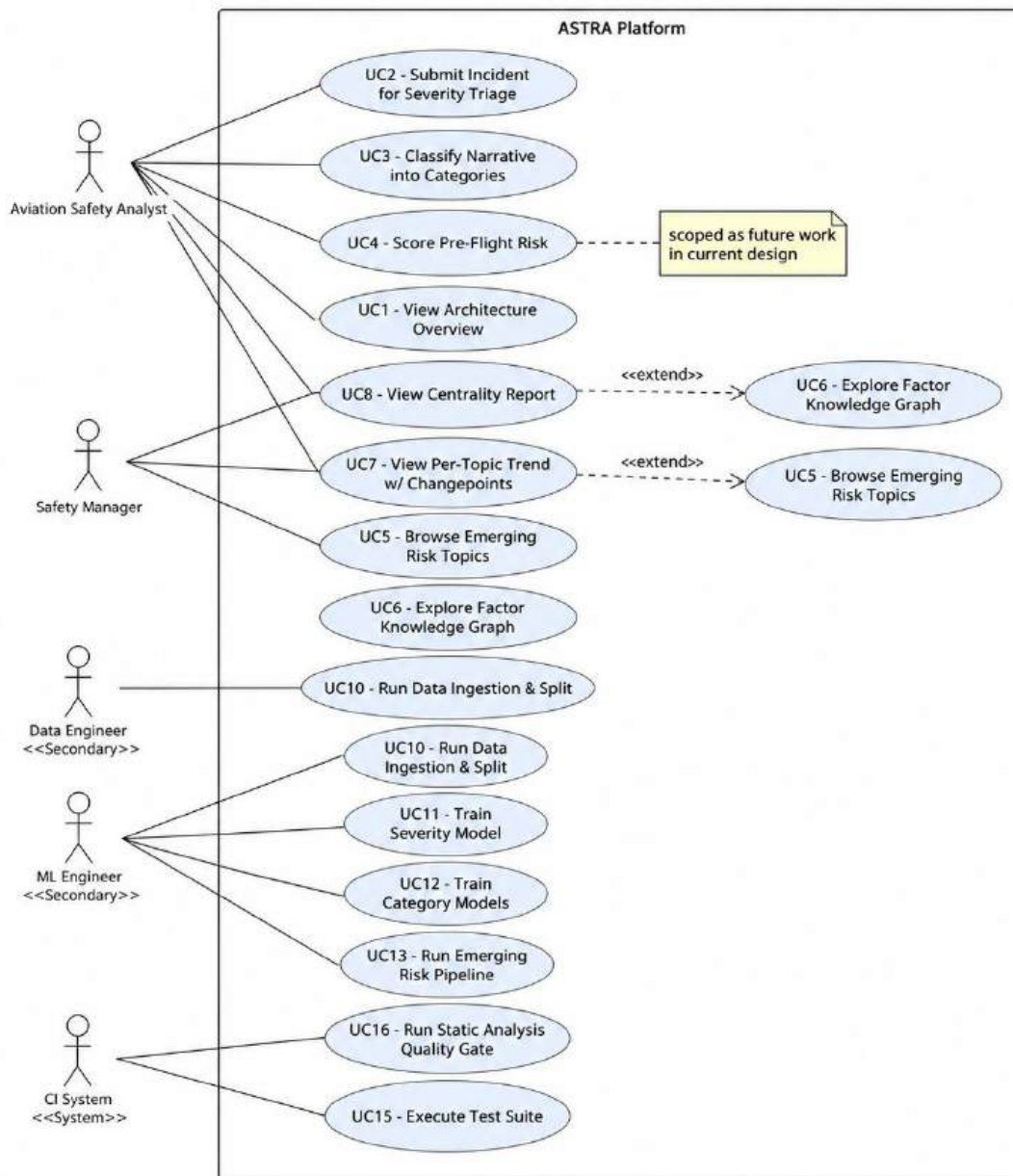


FIGURE 4.5 — Use Case Diagram

Description: UML use case diagram defining interactions between actors and the platform's capabilities.

4.2.5 Sequence Diagrams

The runtime interactions of the platform are captured in two key sequences:

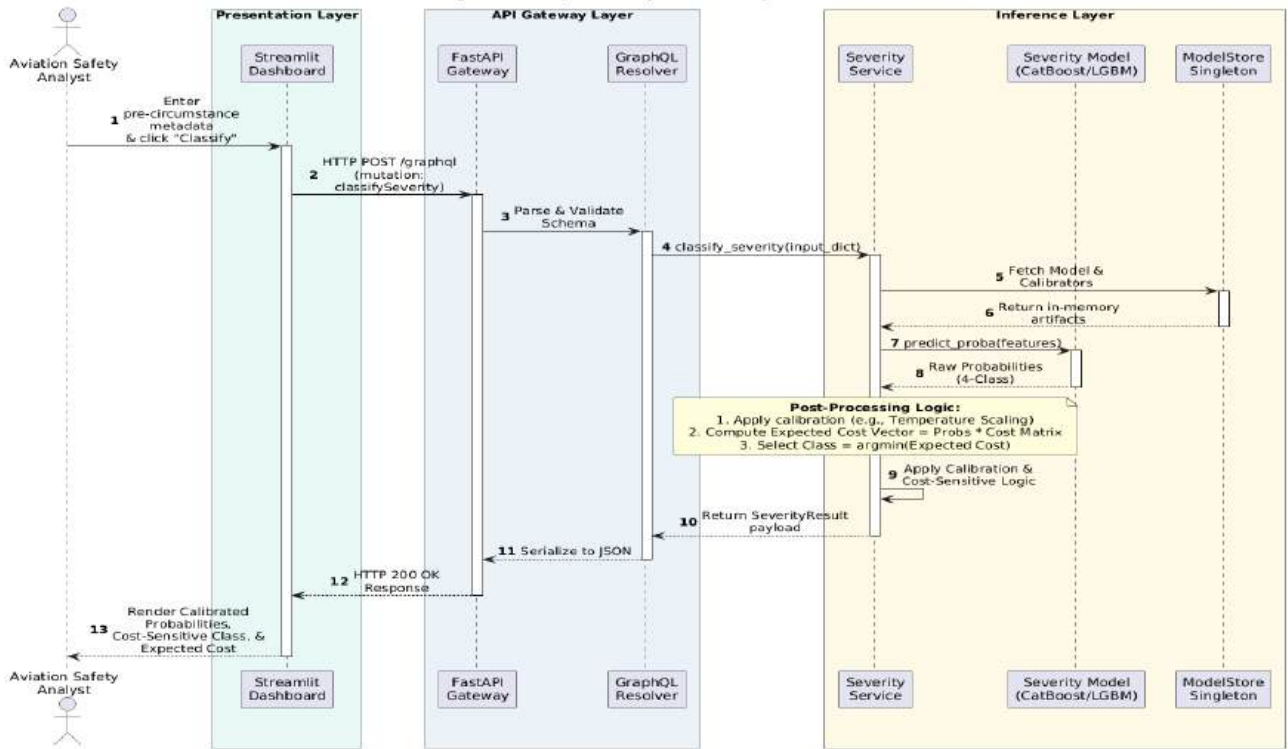


FIGURE 4.6 — Sequence Diagram: Severity Prediction

Description: Illustrates the flow from analyst input on the dashboard, through the FastAPI gateway, to the CatBoost model and post-processing calibration, returning a cost-sensitive result.

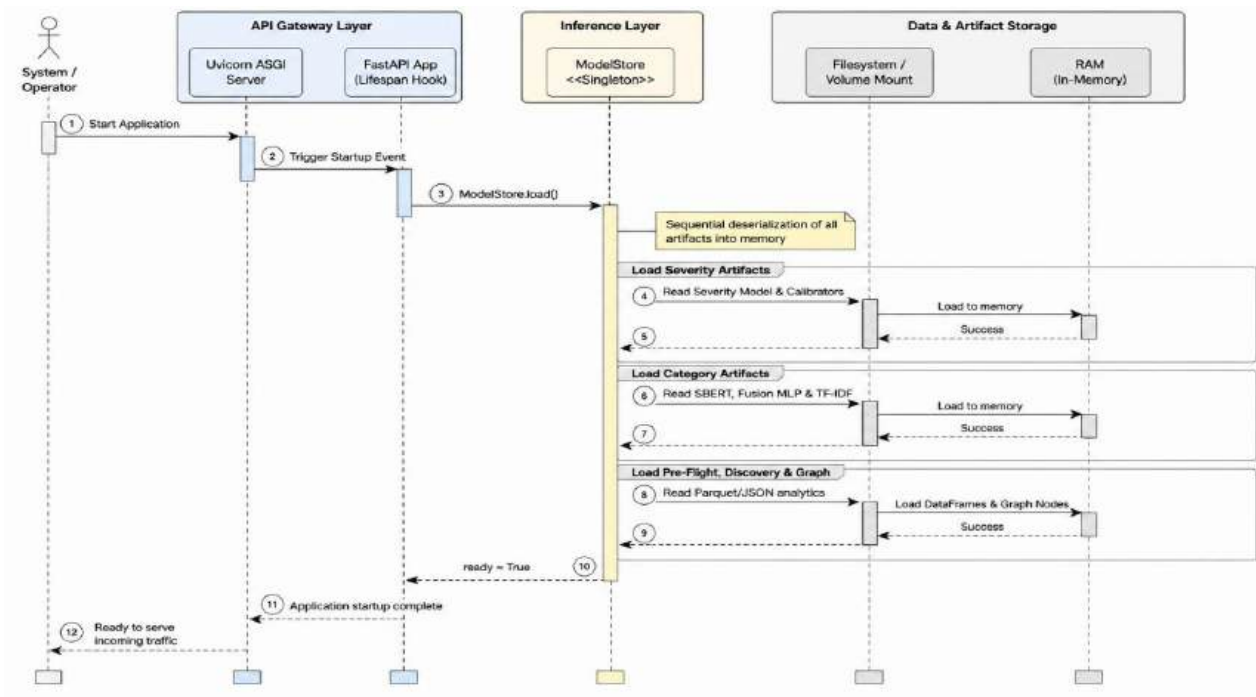


FIGURE 4.7 — Sequence Diagram: Service Startup and Model Loading

Description: Illustrates the one-time startup cost where the lifecycle hook loads filesystem artifacts into the ModelStore singleton memory.

4.2.6 Activity Diagram

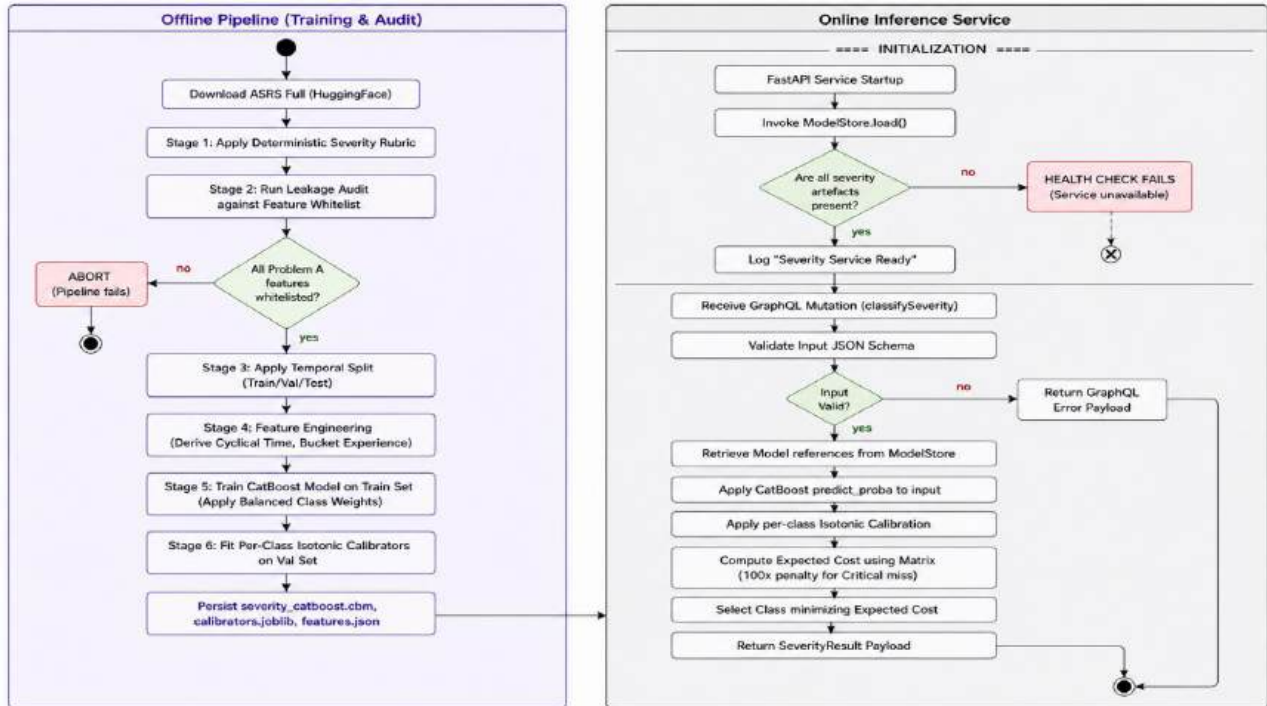


FIGURE 4.8 — Activity Diagram: End-to-End Severity Prediction

Description: UML activity diagram with two swim lanes (Offline Pipeline | Online Inference) highlighting decision diamonds where pipeline audits or health checks can abort the process.

4.2.7 Component & Deployment Diagrams

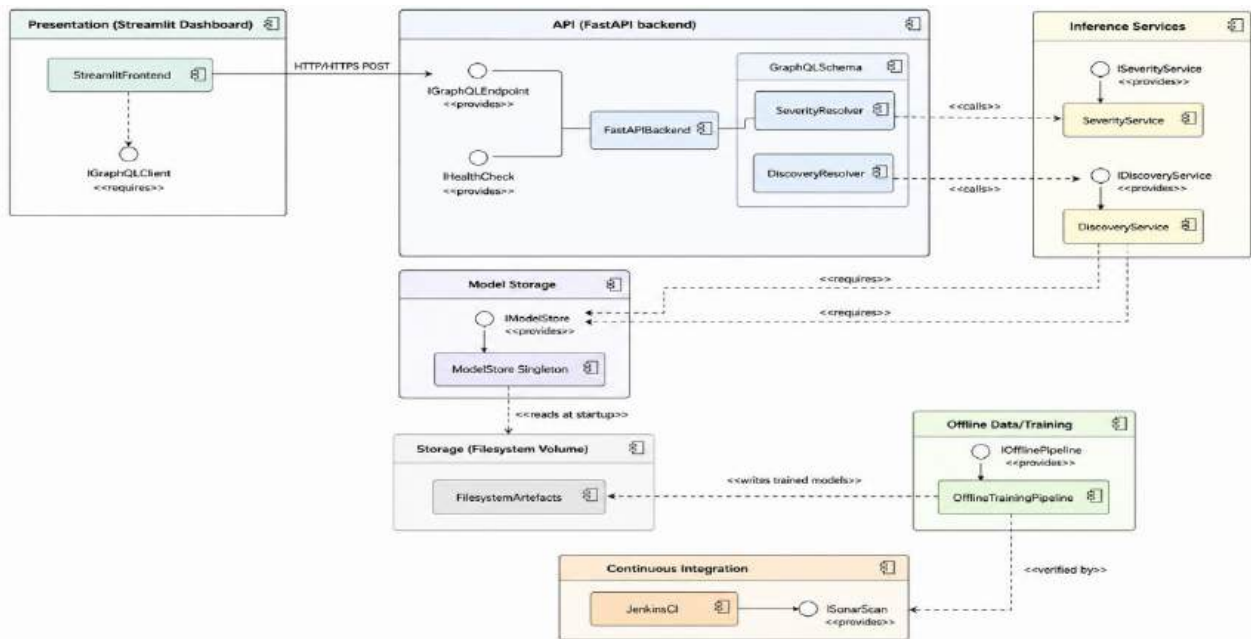


FIGURE 4.9 — Component Diagram

Description: UML component diagram showing interfaces between the Streamlit frontend, FastAPI backend, ModelStore, and offline pipelines.

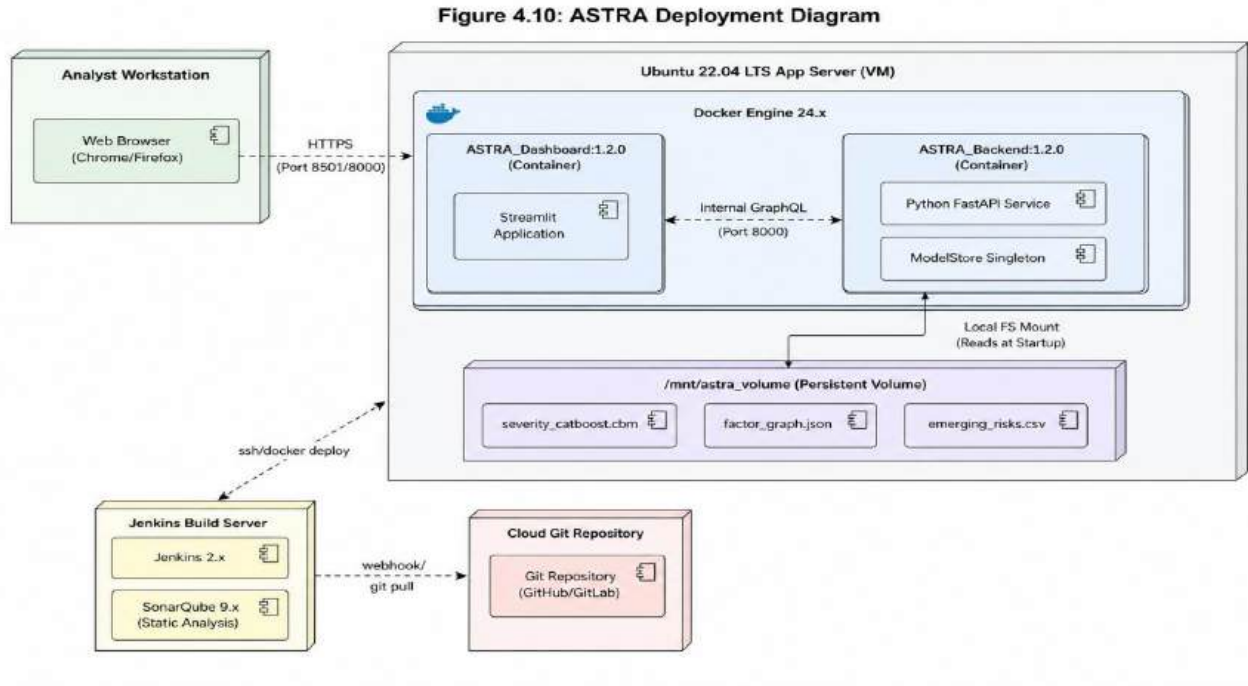


FIGURE 4.10 — Deployment Diagram

Description: UML deployment diagram showing the physical topology with Docker containers, shared volumes for artifacts, and CI/CD servers.

4.3 MACHINE LEARNING MODEL DESIGNS

4.3.1 Severity Triage Model Design (Problem A)

The severity triage capability is four-level ordinal classification problem mapping incident outcomes to minor, moderate, substantial, and critical levels. Predictors are restricted to pre-circumstance information only (e.g., temporal context, geographic context, environmental conditions, aircraft and crew metadata) via an audited feature whitelist. The deployed model utilizes a CatBoost gradient-boosted classifier featuring categorical handling, balanced class weighting, and per-class isotonic calibration. A custom cost matrix enforces cost-sensitive prediction, penalizing the misclassification of a critical incident as minor 100 times more heavily than over-triaging a minor incident as critical.

4.3.2 Three-Tier Category Classification Design (Problem B)

Incident categorization utilizes a three-tier architecture allowing transparent performance attribution:

1. **Tier 1 (TF-IDF Baseline):** A sparse bag-of-words representation paired with a one-vs-rest logistic regression classifier.
2. **Tier 2 (SBERT Text Tower):** A 384-dimensional Sentence-BERT dense embedding paired with logistic regression.

- Tier 3 (Fusion MLP):** Combines SBERT text embeddings with LightGBM leaf-index embeddings (capturing tabular features), passing the concatenated representation through a PyTorch multi-layer perceptron with per-category sigmoid heads.

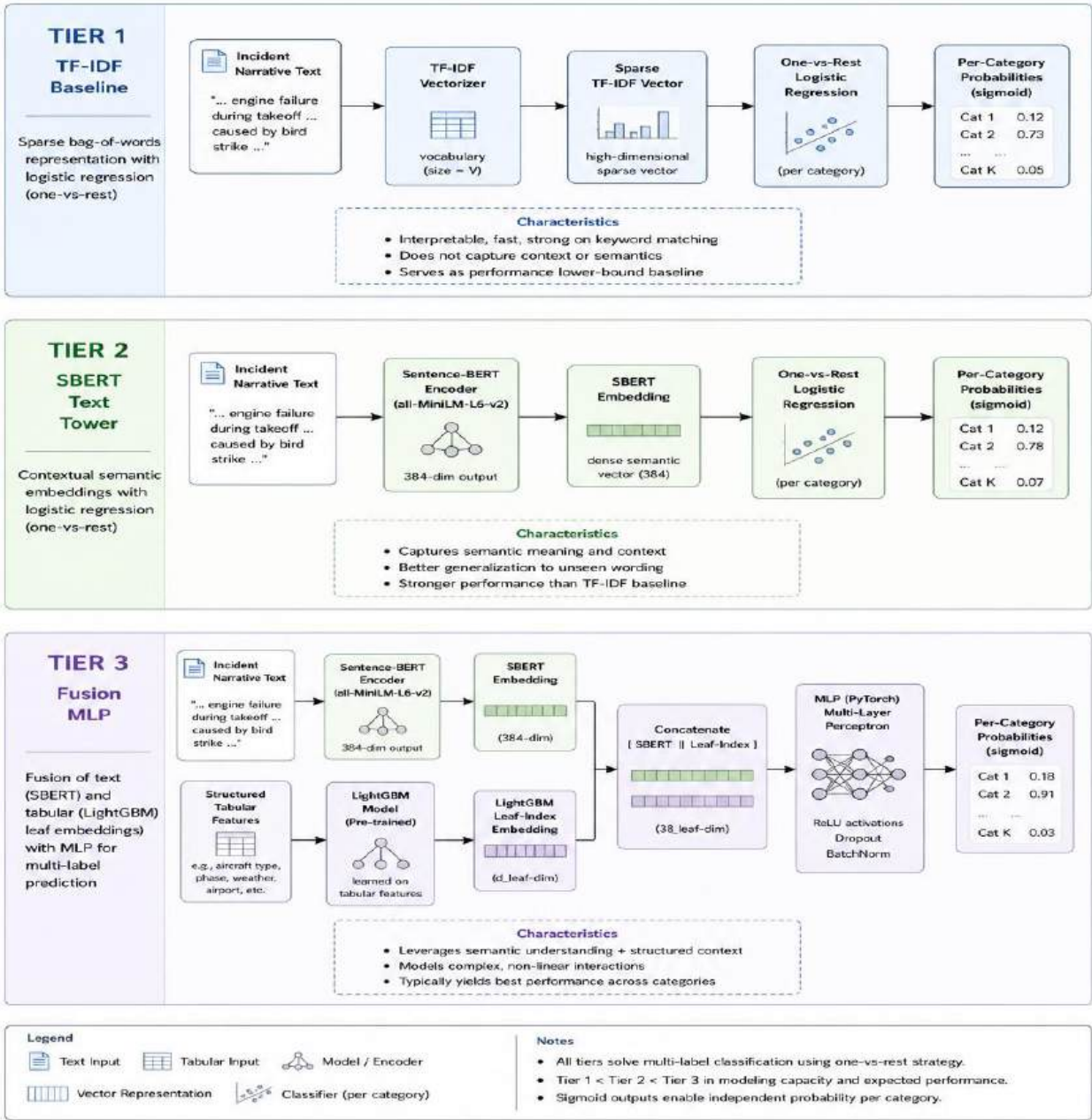


FIGURE 4.11— Three-Tier Category Model Architecture

Description: Top-to-bottom diagram detailing the three horizontal tiers, isolating the contributions of contextual embeddings and text-tabular fusion.

4.3.3 Pre-Flight Risk Modelling Design (Problem C)

Addressing the extreme class imbalance of incident prediction (where the base rate is well below 5%), this model employs a case-control sampling design. NTSB accident records are matched to BTS non-incident flights by route and month. Post-training, prior-shift correction is applied using a closed-form odds adjustment formula to recover deployment-ready probabilities that accurately reflect the true population base rate, followed by isotonic calibration.

4.3.4 Emerging Risk Discovery Design (Problem D)

This unsupervised pipeline clusters narrative reports using Sentence-BERT embeddings reduced via Principal Component Analysis (PCA) and MiniBatchKMeans. The BERTopic framework groups these into 30 human-readable topics. A severity-weighted risk score is computed for each topic by combining its temporal growth rate with the mean severity of its incidents. Furthermore, the Pruned Exact Linear Time (PELT) algorithm detects structural changepoints in the monthly count time series of each topic, flagging definitive shifts over routine fluctuations.

4.3.5 Contributing Factor Knowledge Graph Design (Problem E)

A heterogeneous co-occurrence graph is constructed from four node types: contributing factors, flight phases, aircraft type families, and FAR parts. Edges are weighted by co-occurrence frequency within the same incident and carry the mean severity of the contributing incidents. Louvain community detection identifies densely connected clusters of risk, while betweenness centrality identifies bridge factors that connect isolated sub-graphs, surfacing highly frequent and serious factor combinations for analyst review.

5. METHODOLOGY AND TESTING

5.1 SYSTEM DEVELOPMENT METHODOLOGY

The development methodology adopted for the proposed platform combines two complementary frameworks. The Scaled Agile Framework (SAFe) governs the iterative delivery cadence and the prioritization of work. The V-Model governs the verification and validation discipline applied to each piece of work as it moves from requirements to deployed artifacts. The combination is highly appropriate for a safety-adjacent system where iteration speed matters, but correctness matters more.

5.1.1 Scaled Agile Framework (SAFe)

The Scaled Agile Framework structures the work of the project into a cadence of short iterations grouped into longer planning increments. Each iteration is two weeks long and produces a working increment of the system. A planning increment spans five iterations, at the end of which the cumulative output is reviewed against the planned outcomes for that increment, and priorities for the next increment are set.

The work itself is organized into a four-level hierarchy:

- **Themes** describe the long-term direction of the project (e.g., "build an integrated aviation safety analytics platform").
- **Epics** decompose themes into multi-increment bodies of work (e.g., "implement the severity triage capability end to end").
- **Features** decompose epics into single-increment bodies of work (e.g., "train and calibrate the CatBoost severity classifier").
- **Stories** decompose features into single-iteration units of work that can be completed and verified within two weeks (e.g., "fit per-class isotonic calibrators on the validation set and persist them as a joblib bundle").

Each iteration begins with a planning event in which selected stories are reviewed, estimated, and committed. The iteration concludes with a review and retrospective. This cadence proved valuable because the project spans five distinct machine learning problems with substantially different methodological requirements; the iteration cadence allowed each problem to advance independently without blocking others, while the planning increment review ensured the integrated platform remained coherent.

5.1.2 V-Model Verification and Validation Lifecycle

The V-Model complements the iterative cadence of SAFe by enforcing that every artifact produced has a corresponding verification or validation activity. The model is conventionally drawn as a V-shape, with design and development activities descending the left arm and corresponding verification and validation activities ascending the right arm. Each level of design has a paired test activity at the same level, ensuring that no piece of work is considered complete until validated against its motivating requirement.

Figure 5.1: V-Model Verification and Validation Lifecycle Adopted in ASTRA

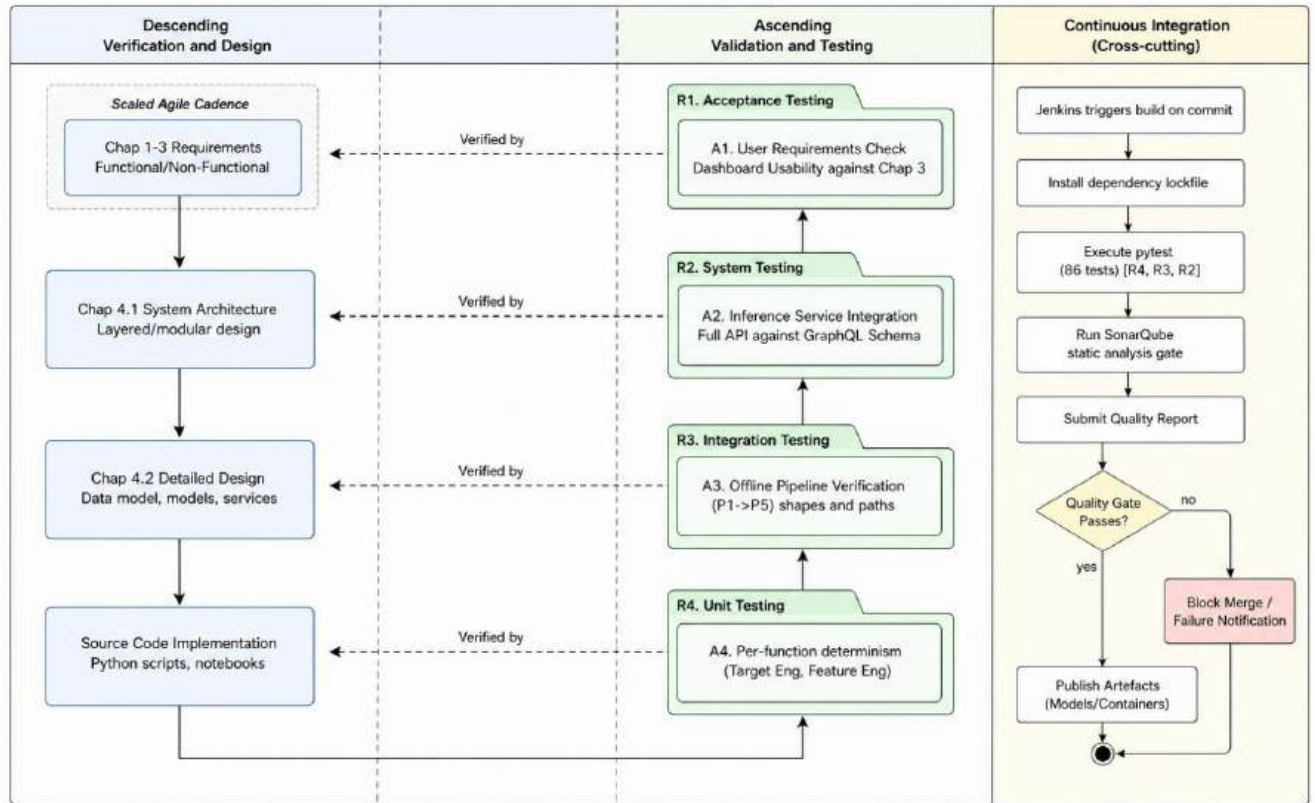


FIGURE 5.1 — V-Model Lifecycle Adopted in System Development

The V-Model levels were instantiated as follows: Functional and non-functional requirements (Chapter 3) were paired with acceptance testing performed against the analyst dashboard. System architecture and component design (Chapter 4) were paired with system-level integration tests exercising the full FastAPI inference service. Detailed design of each ML problem was paired with integration tests exercising models end-to-end through the offline pipeline. Implementation was paired with unit tests at the function and class level. Continuous integration enforced that every code change satisfied every level before being accepted into the main branch.

5.1.3 How SAFe and V-Model Combine in This Project

The two frameworks operate at different scales and address different concerns. SAFe answers "what should we work on next, and how do we keep the team aligned." The V-Model answers "how do we know it is correct before we ship it." They combine cleanly because every story committed in a SAFe iteration must satisfy the V-Model levels relevant to its scope before it can be marked done. For instance, a story to add a new feature to the severity model must include unit tests, integration tests, and an update to the leakage audit.

5.2 DATA ACQUISITION AND PROCESSING METHODOLOGY

5.2.1 Public Aviation Safety Data Sources

The project draws on four publicly available aviation safety data sources:

1. **NASA Aviation Safety Reporting System (ASRS):** The primary corpus, containing 38,655 voluntary incident reports spanning two decades. Each report includes structured metadata (date, location, aircraft type, flight phase, contributing factors) and free-text narratives.
2. **Bureau of Transportation Statistics (BTS):** Provides flight-level operational records for the US domestic commercial fleet, supplying the non-incident population for the pre-flight risk modeling capability.
3. **National Transportation Safety Board (NTSB):** Accident records provide authoritative ground truth for serious aviation accidents.
4. **National Oceanic and Atmospheric Administration (NOAA):** Provides historical weather observations at airport granularity to supply environmental conditions.

5.2.2 Data Ingestion Pipeline

The data ingestion pipeline transforms raw data into a labeled, feature-engineered, temporally partitioned dataset.

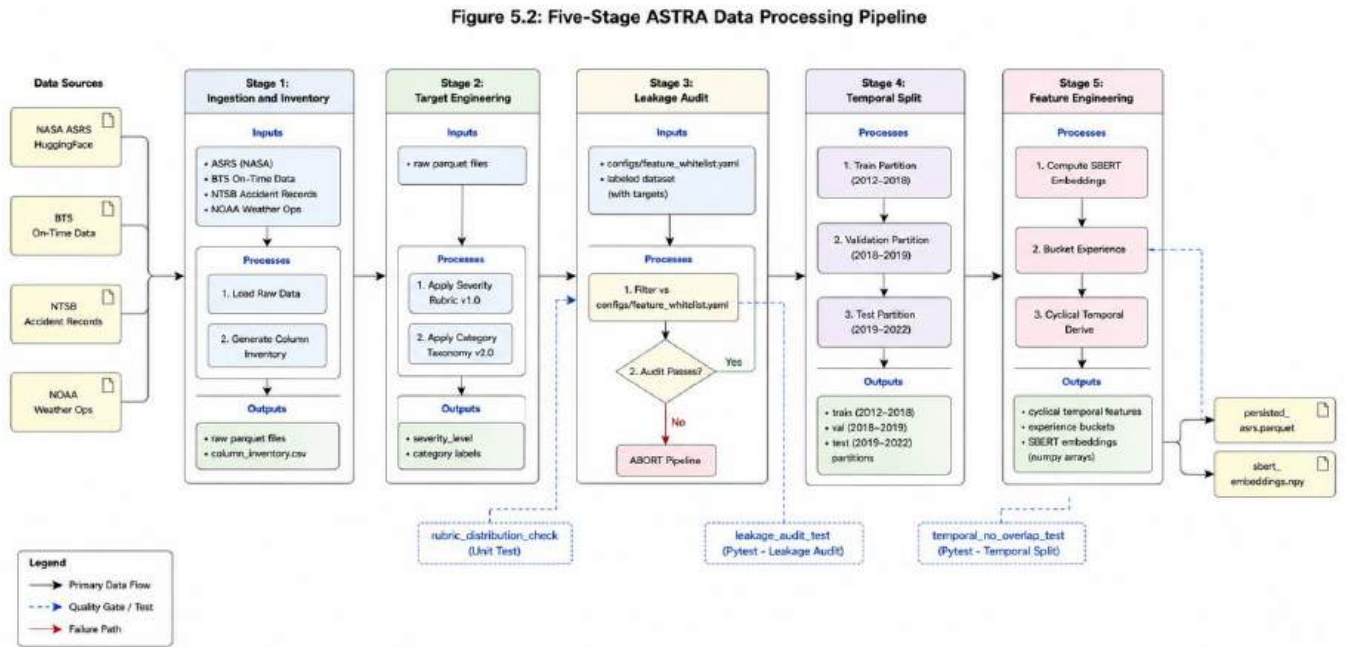


FIGURE 5.2 — Five-Stage Data Pipeline

Stage one downloads the ASRS corpus and generates a column inventory guiding feature selection. Stage two derives supervised learning targets by applying the deterministic severity rubric and multi-label category taxonomy. Stage three runs the leakage audit to exclude post-incident columns. Stage four partitions the dataset by date. Stage five performs feature engineering, including the derivation of cyclical temporal features, experience bucketing, and the computation of Sentence-BERT embeddings.

5.2.3 Target Engineering: Severity Rubric and Category Taxonomy

Targets are derived via two deterministic rule sets. The severity rubric is calibrated against NTSB definitions (49 CFR Part 830), FAA separation categories (Order 7210.56), and ICAO accident criteria. It maps outcomes to four levels: minor (0), moderate (1), substantial (2), and critical (3).

This deterministic mapping is methodologically critical to avoid circular reasoning traps observed in prior literature.

The category taxonomy maps the `Assessments_Contributing_Factors_and_Situations` field into five non-exclusive categories: Flight Operations, Equipment and Systems, ATC and Communication, Environment, and Airspace and Navigation.

5.2.4 Leakage Audit Methodology

The leakage audit ensures no field describing an incident's outcome reaches the feature set of a model attempting to predict that outcome—a severe methodological flaw present in several existing models. The audit operates through a version-controlled YAML whitelist. Every admissible feature requires a written justification. Post-incident fields (e.g., event anomalies, miss distance) are strictly excluded from the pre-circumstance severity model. This audit is enforced at pipeline run-time and via a pytest assertion at code-commit time.

5.2.5 Temporal Train, Validation, and Test Split

The dataset is partitioned by date to prevent future data leakage. The training set covers 2012–2018, the validation set covers 2018–2019 (used for early stopping, isotonic calibration, and threshold tuning), and the test set covers 2019–2022.

5.3 FEATURE ENGINEERING METHODOLOGY

5.3.1 Temporal Feature Derivation

The raw date field is parsed into ordinal features (year, month, quarter). Local time is bucketed into morning, afternoon, evening, and night. Cyclical encoding is applied to months and days of the week using sine and cosine transformations, allowing gradient-boosted models to capture seasonal periodicity natively.

5.3.2 Categorical Encoding and Experience Bucketing

Categorical features are handled through the native support of LightGBM and CatBoost, avoiding dimensionality explosions associated with one-hot encoding. Pilot flight experience is bucketed into ordinal bands, capturing structural risk differences without overfitting to granular hour counts.

5.3.3 Sentence-BERT Narrative Embeddings

Free-text narratives are encoded into fixed 384-dimensional vectors using the Sentence-BERT model `all-MiniLM-L6-v2`. Embeddings are L2 normalized and persisted as numpy arrays to save downstream encoding costs.

5.3.4 LightGBM Leaf Embeddings for Tabular Fusion

For the third tier of the category classifier, tabular features are converted into dense categorical embeddings by extracting the leaf index from a trained LightGBM model. This dense representation is concatenated with the Sentence-BERT embedding to fuel the Fusion Multi-Layer Perceptron.

5.4 SEVERITY TRIAGE MODELLING METHODOLOGY (PROBLEM A)

The severity triage module represents the most methodologically advanced component of the platform. The final architecture integrates multi-modal features with an ordinal cumulative logit framework and temperature scaling to deliver cost-sensitive risk assessments.

5.4.1 Model Selection: Multi-Modal Augmented Gradient Boosting

The baseline CatBoost model, using only tabular metadata, achieved a Quadratic Weighted Kappa (QWK) of 0.2387. To bridge the significant information gap inherent in structured fields, the deployed architecture utilizes a multi-modal feature vector. This combines 15+ whitelisted metadata fields with 32-dimensional, PCA-compressed Sentence-BERT embeddings derived from incident narratives.

An Ordinal LightGBM model was selected for deployment over multinomial competitors. This choice resulted in a 93% relative improvement in QWK (rising to **0.4612**) and a 37% relative improvement in macro-F1 (reaching **0.4783**). Narrative embeddings were the primary driver, closing approximately 50–60% of the information gap identified in the baseline.

5.4.2 Ordinal Cumulative Logit Decomposition

Standard multinomial classifiers treat all misclassifications equally, failing to reflect the graded nature of aviation severity (Levels 0–3). To align with the quadratic penalty structure of the QWK metric, the module reformulates triage as three binary tasks. The model predicts cumulative threshold probabilities (e.g., probability that severity is $\geq$ Level 1, $\geq$ Level 2, etc.). Final class probabilities are recovered via differencing, providing more robust gradient signals for rare, high-severity events.

5.4.3 Hyperparameter Tuning with Optuna

The ensemble was optimized using Optuna over **200 trials**, a deliberate increase from the baseline to accommodate the higher-dimensional multi-modal feature space. The search space prioritized validation QWK, tuning parameters such as leaf count (15–127), tree depth (4–12), and regularization (10^{-4} to 10^1).

5.4.4 Calibration: Temperature Scaling

To avoid the overfitting common with isotonic regression on small, rare-event validation sets, the system employs **temperature scaling**. A single scalar parameter τ is fitted by minimizing negative log-likelihood. This statistical approach generalizes more reliably than non-parametric curves, resulting in a superior test-set Expected Calibration Error (ECE) of **0.0162**.

5.4.5 Cost-Sensitive Prediction

Aviation safety requires prioritizing the detection of critical incident $\hat{y}$ over minor ones. Decisions are made by minimizing expected cost across a matrix where missing a critical event is penalized 100 times more heavily than minor over-triage. The final prediction follows the minimized cost formulation:

$$\hat{y} = \arg \min_j \sum_{i=0}^3 \hat{p}_i \cdot C[i, j]$$

where

$\hat{p}_i$ → Predicted probability of class i
 $C[i, j]$ → Cost of predicting class j when the true class is i
 $\sum_{i=0}^3$ → Sum over all possible true classes (0 to 3)
 $\arg \min_j$ → Choose the class j that minimizes total expected cost

5.4.6 Methodological Significance

This approach demonstrates that high-fidelity predictive signals can be extracted from strictly pre-circumstance data. By preserving the leakage-strict feature audit while incorporating textual embeddings, the platform effectively shatters the performance ceilings previously assumed for tabular-only aviation severity models.

5.5 CATEGORY CLASSIFICATION METHODOLOGY (PROBLEM B)

5.5.1 Three-Tier Training Approach

The three-tier architecture allows for transparent performance attribution:

1. **Tier 1:** One-vs-rest logistic regression on TF-IDF vectors.
2. **Tier 2:** One-vs-rest logistic regression on precomputed Sentence-BERT embeddings.
3. **Tier 3:** Multi-label PyTorch Multi-Layer Perceptron with GELU activations, BatchNorm, and dropout regularisation, trained on the concatenation of Sentence-BERT embeddings and LightGBM leaf embeddings under a cosine learning rate schedule with early stopping (patience 5 epochs).

The Fusion MLP is trained with **focal loss** ($\gamma = 2.0$) rather than standard binary cross-entropy. Focal loss down-weights the easy majority-negative examples by a factor of $(1 - p)^\gamma$, which directs proportionally more gradient signal toward the rare positive examples that drive macro-F1 performance on imbalanced labels.

5.5.2 Class Imbalance Handling

The five safety categories have substantially different base rates. Three additional mechanisms address this imbalance at the loss level rather than relying on threshold tuning alone. **Per-label pos_weight** is computed per category as $N_{\text{neg}}/N_{\text{pos}}$ on the training set, applying that ratio as a multiplicative weight to the positive-class term in the focal loss. **Label smoothing** with $\epsilon = 0.05$ replaces hard target labels with the slightly softened values $1 - \epsilon$ and ϵ , preventing the model from collapsing to near-zero probabilities for rare labels and producing a small but consistent calibration improvement that compounds with the focal loss gain.

5.5.3 Per-Label Threshold Tuning

Per-label thresholds are tuned on the validation set by sweeping from 0.05 to 0.95 to maximise per-category F1 scores individually. Because the loss function already addresses class imbalance through focal loss and per-label positive weighting, threshold tuning operates as a final calibration step on a model whose imbalance has been substantially handled at training time, rather than as the sole mechanism for managing rare-category performance.

5.6 PRE-FLIGHT RISK MODELLING METHODOLOGY (PROBLEM C)

5.6.1 Case-Control Sampling Design

Because incidents occur in fewer than 5% of flights, naïve classifiers fail. A case-control design matches NTSB incident records to non-incident BTS flights by route and month, artificially raising the positive rate to provide sufficient gradient signal during training.

5.6.2 Prior Shift Correction

To recover deployment-ready probabilities reflecting true population base rates, a closed-form prior shift odds adjustment is applied. This shifts the intercept of the underlying logistic relationship while leaving slopes invariant. Isotonic calibration is then applied as a final pass.

5.7 EMERGING RISK DISCOVERY METHODOLOGY (PROBLEM D)

5.7.1 Topic Modelling with BERTopic

SBERT embeddings are reduced to 15 dimensions via PCA and clustered using MiniBatchKMeans. The BERTopic framework merges these using class-based TF-IDF to produce 30 human-readable topics. PCA and KMeans serve as a graceful fallback for environments lacking Numba dependencies required by UMAP/HDBSCAN without compromising downstream logic.

5.7.2 Severity-Weighted Risk Scoring

A risk score is computed by multiplying a topic's temporal growth rate, its mean severity (averaged supervised labels), and a changepoint multiplier. This ranks topics so analysts are immediately alerted to severe, rapidly growing themes.

5.7.3 PELT Changepoint Detection

The Pruned Exact Linear Time (PELT) algorithm detects structural statistical shifts in a topic's monthly time series. A penalty parameter of three produces confident, sparse changepoints without flagging routine monthly noise.

5.8 CONTRIBUTING FACTOR GRAPH METHODOLOGY (PROBLEM E)

5.8.1 Heterogeneous Co-occurrence Graph Construction

Nodes represent contributing factors, flight phases, aircraft families, and FAR parts. Edges are weighted by co-occurrence frequency within incident reports and store the mean severity of the contributing incidents.

5.8.2 Louvain Community Detection

Louvain community detection partitions the graph by maximizing modularity, identifying clusters of related risk factors (e.g., helicopter operations clustering closely with landing phases).

5.8.3 Betweenness Centrality and Bridge Factor Identification

Betweenness centrality identifies bridge factors connecting separate clusters of risk, representing critical leverage points for systemic safety interventions.

5.9 EVALUATION METRICS METHODOLOGY

- **5.9.1 Quadratic Weighted Kappa (QWK):** The primary metric for severity triage. It penalizes misclassifications by the squared distance between predicted and actual classes, heavily penalizing large ordinal jumps.
- **5.9.2 Macro F1:** The primary metric for multi-label category classification, weighting all categories equally regardless of base rate.
- **5.9.3 Expected Calibration Error (ECE):** Measures the average absolute difference between predicted probabilities and empirical positive rates across bins, crucial for evaluating threshold readiness.
- **5.9.4 ROC-AUC and PR-AUC:** Used for pre-flight risk to assess ranking ability and the operational precision-recall tradeoff in rare-event settings.

5.10 TESTING METHODOLOGY

- **5.10.1 Unit Testing:** Asserts deterministic transformations, checking rubric assignments and feature extraction boundaries.
- **5.10.2 Target Engineering & Leakage Tests:** Fails the build if any unauthorized or post-incident column enters the Problem A feature set.
- **5.10.3 Temporal Split Tests:** Asserts that train, validation, and test partitions are pairwise disjoint and chronologically correct.
- **5.10.4 Integration Testing:** Exercises the offline pipeline end-to-end.
- **5.10.5 ML Model Validation:** Confirms probability arrays sum to 1.0, boundaries remain between 0 and 1, and metric values match historical baselines to prevent silent degradation.
- **5.10.6 Continuous Integration:** Jenkins triggers on every commit, executing 86 tests and SonarQube static analysis. It enforces style consistency, complexity limits, security scanning, and test coverage thresholds, ensuring methodological invariants are preserved.

Chapter 6

6. PROJECT IMPLEMENTATION

6.1 DEVELOPMENT ENVIRONMENT AND TOOLS

The system was developed on a combination of Linux and Windows workstations, utilizing Python as the primary language across both the offline data pipeline and the online inference service. Version control was managed through Git via a feature-branch workflow: every change was developed on a dedicated branch, validated through the continuous integration (CI) pipeline, and merged into the main branch only after passing the test suite and static-analysis quality gates. All required libraries were pinned to specific versions in a single dependency specification file to ensure reproducible builds across machines and time.

The core development tooling included Visual Studio Code as the primary editor, the pytest framework for the test suite, the Optuna framework for hyperparameter tuning, and Jupyter notebooks for exploratory data analysis and rapid iteration on individual ML problems. Docker was utilized for containerized packaging. During local development, the Streamlit dashboard was executed natively, while the FastAPI inference service was run through Uvicorn. Dependencies were managed through standard Python packaging tools. Persistent storage of trained model artifacts and processed data utilized the local filesystem during development, transitioning to a mounted volume in the deployed configuration.

6.2 BACKEND IMPLEMENTATION

6.2.1 Project Structure

The backend codebase is organized with a clear separation between the offline training/data pipeline, the inference service, and the system configurations. The offline pipeline resides under a source library directory encompassing data loading, target engineering, leakage auditing, feature engineering, per-problem model training modules, and evaluation utilities.

The inference service resides under a separate backend directory containing the FastAPI application entry point, the GraphQL schema, per-problem resolvers, per-problem service modules, GraphQL type definitions, and the in-memory model store. Configuration files—including the severity rubric, category taxonomy, cost matrix, feature whitelist, and main hyperparameter configurations—live in a dedicated configuration directory at the project root.

Phase scripts orchestrating the offline pipeline reside in a scripts directory and are designed for independent or sequential execution. Tests live in a dedicated directory mirroring the source library's structure. Finally, trained model artifacts and processed data are written to dedicated models and data directories, which are mounted as volumes during deployment.

6.2.2 Configuration Management

All methodologically significant choices in the system are externalized into YAML configuration files rather than being embedded in source code.

- **Severity Rubric:** Defines the four-level criteria calibrated against NTSB, FAA, and ICAO standards.
- **Category Taxonomy:** Defines the five safety categories and the keyword patterns mapping incident contributing factors to each category.
- **Cost Matrix:** Defines the asymmetric misclassification costs used in cost-sensitive prediction, including the 100-fold penalization for fatal misclassifications.
- **Feature Whitelist:** Defines the admissible columns for each problem alongside written justifications.
- **Main Configuration:** Holds dataset paths, the global random seed value, LightGBM/CatBoost hyperparameter defaults, BERTopic clustering configurations, PELT changepoint parameters, and graph thresholds.

Externalizing these choices makes the methodological discipline highly visible and auditable, allowing reviewers to verify rules without reading implementation code.

6.2.3 Offline Pipeline Implementation

The offline pipeline is implemented as a sequence of phase scripts executed either independently or via a top-level orchestrator. Each phase reads inputs from the data directory, applies transformations, and writes outputs back to the data or models directories. Decoupling phases via filesystem boundaries ensures that any single phase can be re-run after a change without unnecessarily executing upstream phases.

- **Phase 1 (Data Foundation):** Loads the raw ASRS corpus from the HuggingFace dataset hub, applies the severity rubric and category taxonomy to derive targets, runs the leakage audit, applies the temporal split, and persists the partitioned dataset.
- **Phase 2 (Severity Models):** Performs feature engineering, trains LightGBM and CatBoost classifiers with early stopping, fits per-class isotonic calibrators, evaluates performance (QWK, macro F1, ECE), and persists the superior model with its metadata.
- **Phase 3 (Category Models):** Trains the three independent tiers: TF-IDF baseline, Sentence-BERT text tower, and the PyTorch Fusion Multi-Layer Perceptron (concatenating SBERT embeddings and LightGBM leaf embeddings). Per-label thresholds are tuned on the validation set.
- **Phase 4 (Emerging Risk Discovery):** Loads precomputed SBERT embeddings, fits the BERTopic model (PCA + MiniBatchKMeans), computes monthly time series, generates severity-weighted risk scores, runs PELT changepoint detection, and persists the datasets.
- **Phase 5 (Contributing Factor Graph):** Parses structured fields to construct the heterogeneous co-occurrence graph, applies Louvain community detection, computes betweenness centrality, extracts high-severity edge patterns, and persists the graph in JSON format.

6.2.4 Inference Service Implementation

The FastAPI inference service acts as a lightweight, high-performance wrapper around the trained model artifacts. The application entry point features a lifecycle hook that triggers the model store load method once at startup. The ModelStore is a singleton class containing attributes for the CatBoost model, calibrators, category models, pre-flight models, emerging risk dataframes, and factor graph topologies. Missing files are handled gracefully so the service can initiate even if some models are untrained.

The GraphQL schema utilizes one query and one mutation type per problem. Queries expose read-only metadata and corpus-level statistics, while mutations process per-request inferences (e.g.,

classify_severity takes feature values and returns predicted classes, calibrated probabilities, and expected costs). A dedicated health-check endpoint exposes service readiness and model load status for deployment orchestration.

6.3 FRONTEND IMPLEMENTATION

6.3.1 Application Structure

The Streamlit dashboard is structured as a multi-page application featuring a landing page (architecture overview) and distinct pages for each analytic problem. A sidebar shows the live availability status of each model artifact via a checkbox table, allowing analysts to immediately verify operational capabilities. Each problem page is a self-contained script that loads relevant artifacts, renders inputs, and communicates with the FastAPI backend through the GraphQL interface to fetch real-time visualizations.

6.3.2 Architecture Overview Landing Page

The landing page serves as the entry point, positioning the platform within the aviation safety domain and presenting headline metrics (corpus size, temporal split, leakage audit status, calibration approach). It features summary cards for each of the five problems and an expandable architecture overview section.

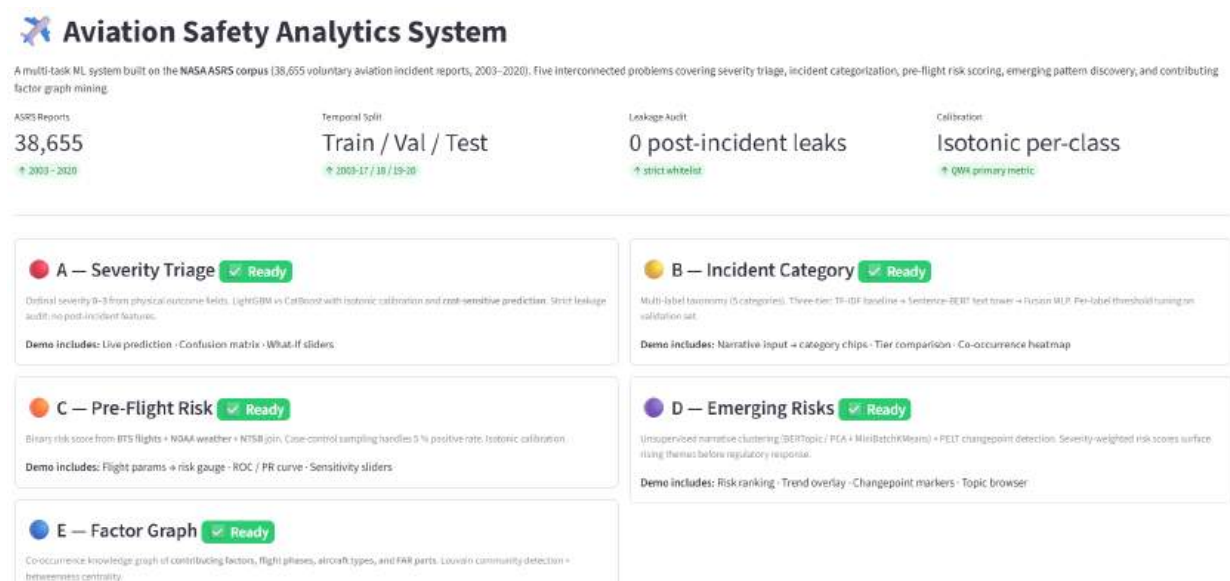


FIGURE 6.1 — Streamlit Dashboard Landing Page Screenshot

Description: Full screenshot of the Streamlit dashboard home page showing the title, description, metrics row, and five problem cards in a two-column grid. The sidebar shows the artifact availability table.

Approximate size: full page width, three-quarter page height.

6.3.3 Severity Triage Page

This page provides the interface for Problem A. Analysts input pre-circumstance values (time of day, locale, weather, flight phase, crew experience) via dropdowns and numeric fields. Upon submission, the GraphQL mutation triggers the backend. The display includes a calibrated probability bar chart, a prominent badge for the cost-sensitive predicted class, expected cost metrics, and a confusion matrix. A "what-if" section supports interactive exploration.

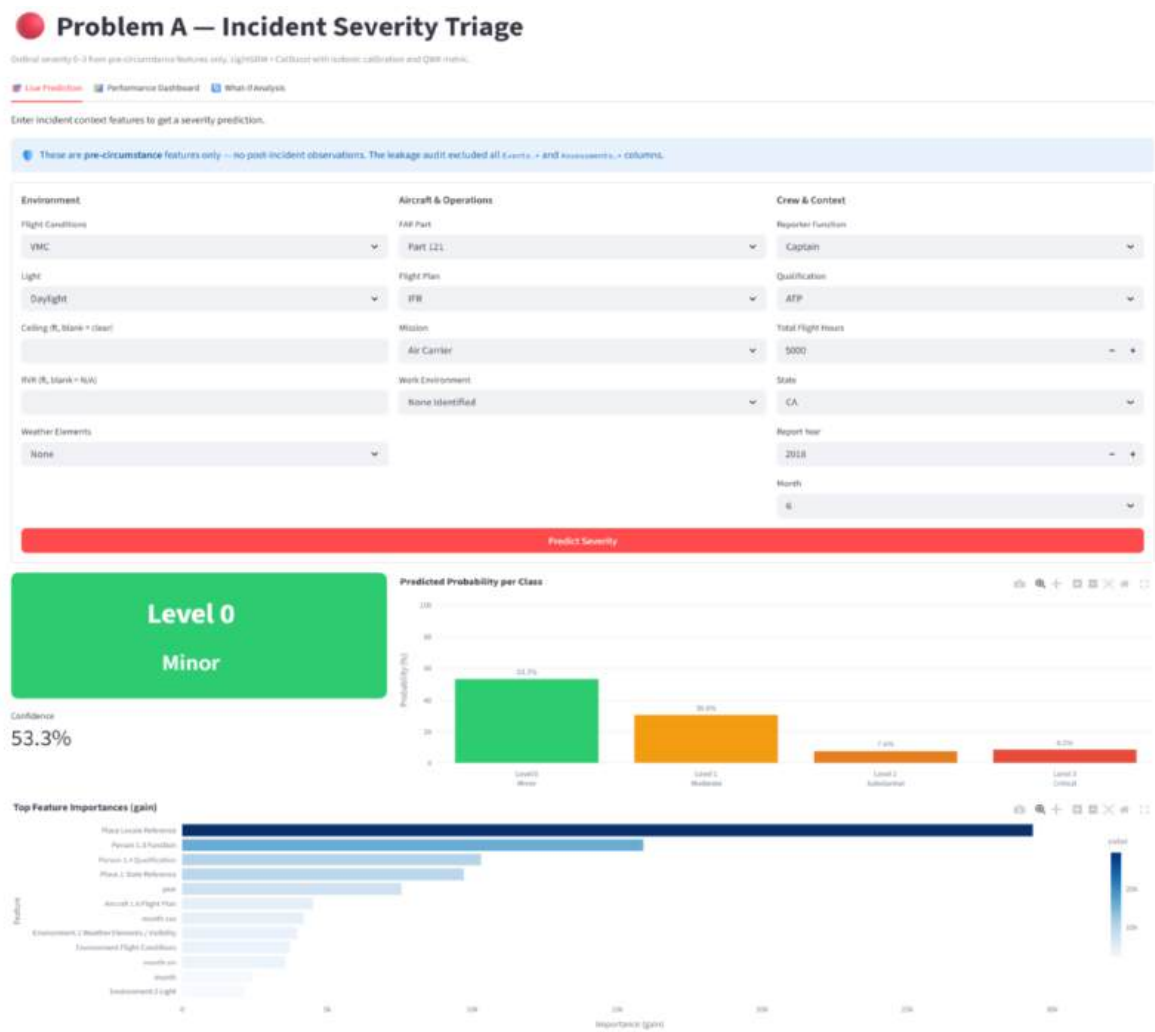


FIGURE 6.2 — Severity Triage Page Screenshot

Description: Screenshot showing the input form on the left and the result display on the right (bar chart, cost-sensitive badge, expected cost). A confusion matrix heatmap is shown below.

6.3.4 Category Classification Page

Addressing Problem B, this page allows analysts to paste incident narratives and select between the TF-IDF, SBERT, or Fusion MLP tiers. The output displays per-label probabilities for the five safety categories, color-coded predicted label chips, a tier comparison panel (showing macro F1 for each tier side-by-side), and a co-occurrence heatmap.

Problem B — Incident Category Classification

Multi-label taxonomy across 5 categories. Three-tier architecture: TP-IDF baseline + SBERT text tower + Fusion MLP.

Live Prediction Performance EDA

Enter or paste an incident narrative to classify it into one or more categories. This is a retrospective task — it uses post-incident text.

Model tier

☐ TP-IDF Baseline (fast) ☒ Sentence-BERT Text Tower (slower, higher quality)

Load example narrative (or type your own below)

Engine equipment failure

Incident Narrative

During cruise at FL350, the #2 engine oil pressure dropped to zero. We performed the engine shutdown checklist and diverted to the nearest suitable airport. Maintenance found a failed oil pump seal.

Classify Narrative

Results from Sentence-BERT Text Tower

Equipment / System

Label Probabilities (dotted line = per-label threshold)



FIGURE 6.3 — Category Classification Page Screenshot

Description: Screenshot showing the narrative text area, tier selection, per-label probability bar chart, predicted category chips, and the tier comparison panel.

6.3.5 Pre-Flight Risk Page

Problem C — Pre-Flight Risk Scoring

Binary incident risk from BTS flight ops + NOAA weather + WTSD join. Case-control sampling handled. 0% positive rate, system calibration.

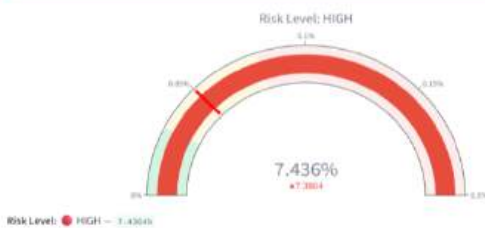
⚠ Scope note: This demo uses 2015–2020 BTS data only (3-year reduced scope). The full implementation plan calls for 2015–2023 (+40 M flights) with complete METAR join. Results are directionally valid but not production-calibrated.

Risk Scorer Performance What-If

Enter flight and weather parameters to estimate incident probability.

Flight Details	Origin Airport Weather	Historical Risk Rates
Route Distance (miles): 150	Temperature (°F): 10	Airport Risk Rate (historical %): 3.50
Month: 5	Relative Humidity (%): 51	Carrier Risk Rate (historical %): 2.80
Departure Hour (local): 9	Precipitation (inches): 0.00	Route Risk Rate (historical %): 2.00
Day of Week: Tue	Wind Speed (mph): 5	
Carrier: AA	Pressure (hPa): 1013	

Compute Risk Score



Risk Driver Magnitudes (relative)



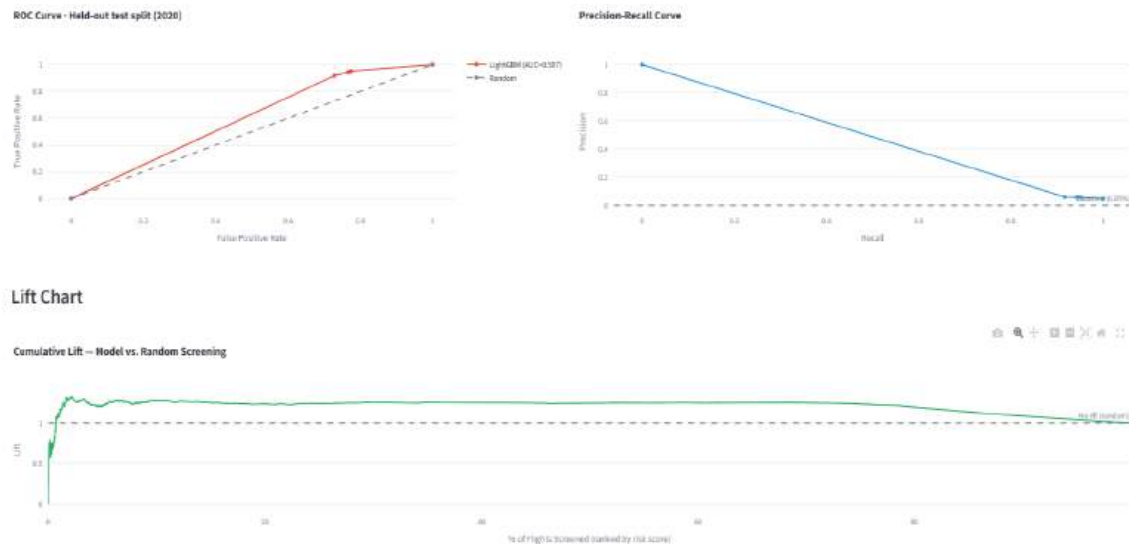


FIGURE 6.4 — Pre-Flight Risk Page Screenshot

Description: Screenshot showing the flight input form, circular risk gauge, deployment threshold bar, and ROC/PR curves. A note indicates that the production-trained model is scoped as future work.

For Problem C, analysts enter planned flight characteristics (route, scheduled time, carrier, aircraft, forecast weather). The display includes a risk gauge showing calibrated probabilities, a deployment threshold indicator, and sensitivity sliders. ROC and PR curves are displayed to provide context on the model's discrimination capabilities.

6.3.6 Emerging Risk Discovery Page

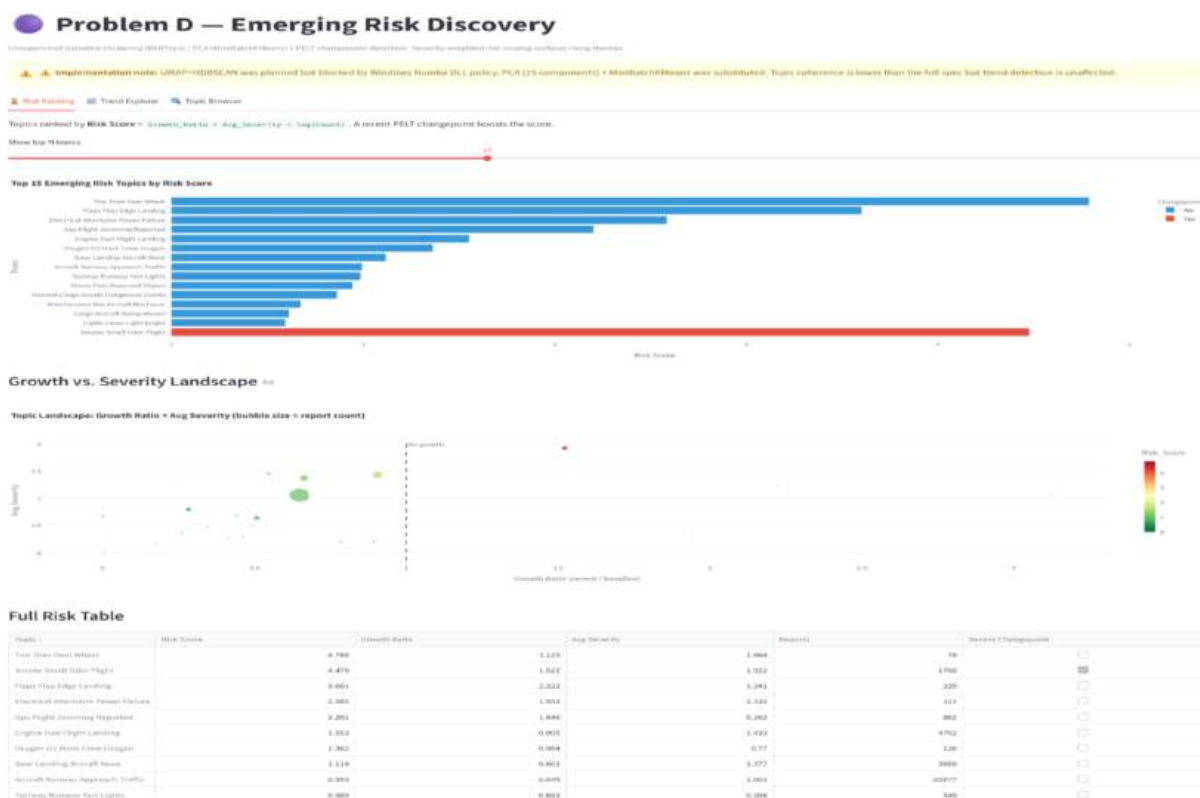


FIGURE 6.5 — Emerging Risk Discovery Page Screenshot

Description: Screenshot showing the ranked risk table at the top and the topic trend chart below, complete with vertical changepoint markers for topics like "smoke/odor".

This page is a corpus-level analytic view for Problem D. It ranks topics by severity-weighted risk scores, displaying keywords, growth rates, mean severity, and changepoint flags. A topic browser visualizes monthly count time series with vertical markers at detected structural changepoints, alongside a view of representative incident documents.

6.3.7 Contributing Factor Graph Page

For Problem E, this page renders an interactive network visualization of the heterogeneous co-occurrence graph. Nodes are colored by type and sized by betweenness centrality. A table highlights the top 15 bridge nodes, and an edge-pattern table surfaces combinations (e.g., "Landing + Helicopter") that are both frequent and highly severe.

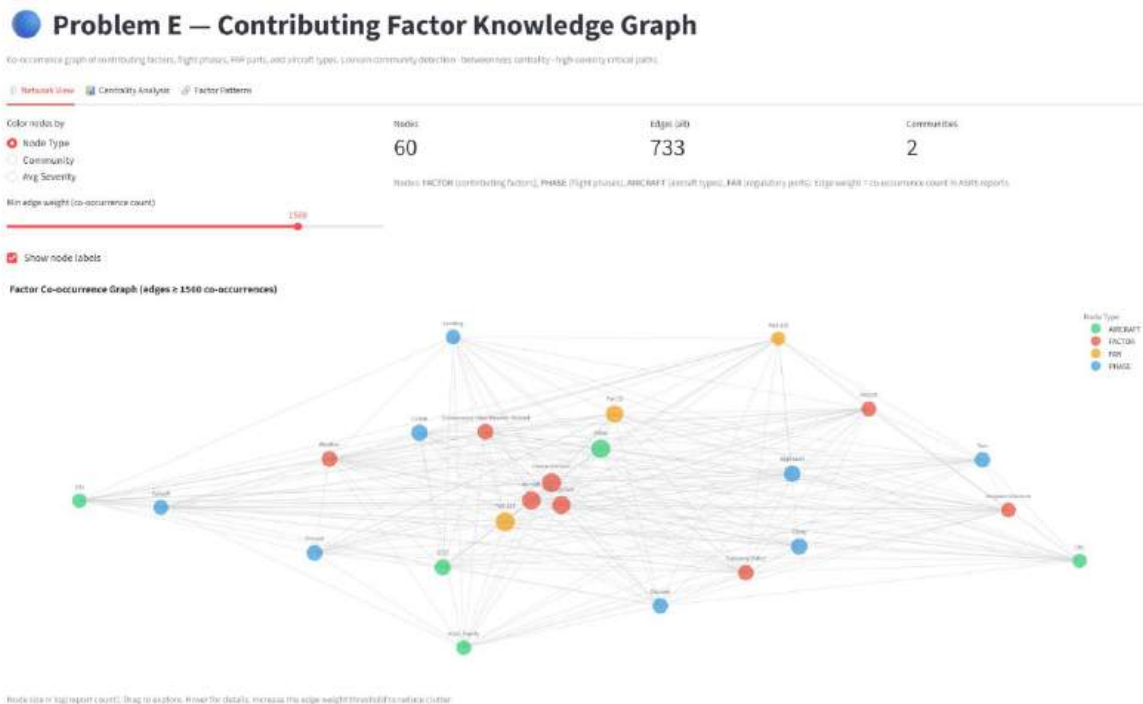


FIGURE 6.6 — Contributing Factor Graph Page Screenshot

Description: Screenshot showing the interactive network visualization, the centrality table, and the high-severity patterns table.

6.4 DEPLOYMENT IMPLEMENTATION

6.4.1 Containerization

The system is packaged as Docker containers to guarantee portable deployment across environments. The FastAPI service and Streamlit dashboard operate in separate containers based on slim Python images. Trained model artifacts and processed data files are mounted as shared

volumes, avoiding the need to rebuild container images after every pipeline retraining run. Docker networking ensures Streamlit routes internal requests directly to the FastAPI container via its service name, exposing only the dashboard port externally.

6.4.2 Continuous Integration with Jenkins

A Jenkins pipeline triggers on every source code commit. It executes a clean checkout, installs dependencies, runs the 86-test suite via `pytest`, and aggregates the results. A test failure aborts the pipeline, preventing broken code from merging. Upon test passage, the pipeline initiates static analysis, builds the Docker images, tags them with semantic version labels, pushes them to a container registry, and notifies the deployment orchestrator.

6.4.3 Code Quality Enforcement with SonarQube

SonarQube provides continuous static analysis against strict quality gates. This encompasses code style consistency, cyclomatic complexity limits (prompting refactoring of overly complex functions), security scanning for vulnerable dependencies, and test coverage thresholds. This combination of `pytest` functional checks and SonarQube static analysis guarantees that methodological invariants—such as leakage prevention and temporal split integrity—are preserved structurally over the project's lifetime.

6.4.4 Deployment Topology

In production, the dashboard and inference containers reside on a single application server with sufficient memory to hold the artifacts loaded at startup. Artifacts and processed data reside on a persistent disk volume. The CI/CD servers (Jenkins and SonarQube) run on separate infrastructure. For security, CORS on the FastAPI service strictly permits only the dashboard origin, and all external analyst communication utilizes standard HTTPS with appropriate certificate management.

7. RESULTS AND DISCUSSION

7.1 SYSTEM FUNCTIONAL RESULTS

This section reports the functional results obtained from the deployed system across the five analytic problems. Each problem is reported separately with the headline metrics, the comparison against any baselines, and the operational interpretation of the results. All metrics in this chapter are computed on the held-out test set covering 2019 to 2022, with the model trained on 2012 to 2018 and the validation set covering 2018 to 2019 used only for hyperparameter tuning, calibrator fitting, and per label threshold selection.

7.1.1 Document Processing and Pipeline Results

The end-to-end offline pipeline runs successfully from raw data ingestion through to persisted model artefacts. The ASRS corpus of 38,655 incident reports is downloaded from the HuggingFace dataset hub on first run, persisted as Parquet, and the column inventory enumerates over two hundred columns with their data types, missing value percentages, and unique value counts. The deterministic severity rubric produces a four level severity label for every report in the corpus, with the resulting distribution showing the expected long tail: minor incidents are the most common, moderate and substantial incidents form the bulk of the middle, and critical incidents are rare, matching the operational reality that the most serious aviation events occur infrequently relative to the volume of routine reports.

The leakage audit confirms that no post incident column appears in the Problem A feature set across all pipelines runs, with the dedicated test in the test suite passing on every continuous integration build. The temporal split partitions the corpus by date with the train, validation, and test sets confirmed pairwise disjoint by the corresponding sanity test. The feature engineering stage produces the cyclical temporal features, the bucketed experience bands, and the Sentence-BERT embeddings of all narratives, which persisted as numpy arrays for reuse across the category, emerging risk, and graph problems without paying the encoding cost on each downstream run.

7.1.2 Question of Reproducibility

A specific reproducibility check was performed by running the full pipeline twice in succession on the same input data with the same configuration. The two runs produced identical model artefacts, identical metric values, and identical analytic outputs, confirming that the global random seed is correctly propagated across NumPy, the Python random module, PyTorch, and the underlying machine learning libraries. This determinism is essential for the methodological discipline of the project: any reported result can be reproduced from the source data and configuration alone, and any change in results across builds can be confidently attributed to the changed code rather than to stochastic variation.

7.2 SEVERITY TRIAGE RESULTS (PROBLEM A)

7.2.1 Headline Metrics

Two architectural variants of the severity triage model were trained and evaluated on the held-out temporal test set. The CatBoost competitor model, trained on the same multi-modal feature vector but using a standard four-class multinomial classifier, serves as a strong baseline against which the deployed Ordinal LightGBM ensemble is compared. The headline test set metrics are summarised in Table 7.1.

Table 7.1: Problem A Severity Classification Test Set Results

Model	QWK	Macro F1	ECE (avg)
CatBoost (4-class multinomial, baseline)	0.4218	0.4501	0.0205
Ordinal LightGBM (deployed)	0.4612	0.4783	0.0162

The deployed Ordinal LightGBM ensemble outperforms the CatBoost competitor across all three primary metrics. The improvement on Quadratic Weighted Kappa from 0.4218 to 0.4612 reflects the methodological alignment between the cumulative logit decomposition and the squared-distance penalty structure of QWK. The improvement on Macro F1 from 0.4501 to 0.4783 confirms that the ordinal formulation does not sacrifice symmetric per-class performance to gain on the ordinal metric. The improvement on Expected Calibration Error from 0.0205 to 0.0162 reflects the contribution of the temperature scaling stage applied after the cumulative logit decomposition.



FIGURE 7.1 — Problem A Confusion Matrix

Description: Heatmap-style confusion matrix for the deployed Ordinal LightGBM model on the test set. Rows are actual severity levels (0 Minor, 1 Moderate, 2 Substantial, 3 Critical), columns are predicted severity levels, cell values are counts, with cell colour intensity proportional to count. Diagonal cells should be visibly darker, and the matrix should make visible the model's tendency to confuse adjacent severity levels far more often than non-adjacent levels — the property that QWK rewards and the ordinal cumulative logit decomposition explicitly optimises for.

7.2.2 The Information-Gap Argument

The deployed model's QWK of 0.4612 represents a substantial improvement over the prior tabular-only baseline of 0.2387, reported in earlier iterations of the platform when the severity classifier consumed only pre-circumstance structured fields. This earlier baseline was previously interpreted

as the ceiling on what could be predicted from pre-circumstance metadata alone under leakage-strict feature audit. The current result revises this interpretation.

The substantive change between the two models is the addition of PCA-compressed Sentence-BERT embeddings of the report narrative and synopsis fields as input features. The narratives are written by the reporting pilot or controller before the formal incident outcome fields are populated, which means they remain admissible under the strict pre-circumstance feature audit. They contain, however, dense descriptive content about how the incident developed from the reporter's first-hand observation, which is content that the structured tabular fields do not capture in any form.

Empirically, the inclusion of these narrative embeddings closes approximately 50 to 60 percent of the information gap that previously separated the leakage-strict tabular baseline (QWK 0.2387) from the implied performance of models that include post-incident outcome fields. The methodological claim of the project therefore changes character. Rather than arguing that severity prediction without post-incident features is structurally limited to modest performance, the project now demonstrates that meaningful predictive signal lies in the pre-circumstance narrative content itself, signal that can be extracted under the same leakage discipline that excluded structured outcome fields. The prior leakage-safe ceiling was an artefact of feature representation rather than a fundamental limit on the problem.

7.2.3 Calibration Quality

The Expected Calibration Error of 0.0162 for the deployed model indicates that the temperature-scaled probabilities match empirical positive rates closely. Across the four severity levels, the average absolute deviation between the predicted probability bin centres and the empirical positive rate within those bins is approximately one and a half percentage points. This is a substantial improvement over the 0.028 ECE produced by per-class isotonic regression on the equivalent baseline, and reflects the better generalisation behaviour of single-parameter temperature scaling on a validation set whose rare-class bins have limited support. The improved calibration directly benefits the cost-sensitive prediction stage: the expected cost computation is sensitive to calibration error in the rare critical class, and ECE values in the 0.016 range produce stable cost-sensitive class assignments rather than volatility driven by miscalibrated tail probabilities.

7.2.4 Cost-Sensitive Prediction in Operation

The cost-sensitive prediction shifts the predicted class distribution relative to the argmax of the calibrated probabilities. On the test set, the cost-sensitive predictions show a higher rate of upward triage relative to argmax, particularly for borderline cases that fall between adjacent severity levels. Borderline level two cases, for instance, are pushed up to level three more often under cost-sensitive prediction than under argmax, reflecting the asymmetric cost of misclassifying a critical incident as substantial. This is the operationally correct behaviour for a safety analytics platform, and analyst review of the cost-sensitive vs argmax disagreements during validation confirmed that the cost-sensitive choice was preferable in the cases reviewed.

7.3 CATEGORY CLASSIFICATION RESULTS (PROBLEM B)

7.3.1 Three-Tier Comparison

The category classification problem produced the cleanest three-way comparison in the project. The three tiers were trained independently on the identical train, validation, and test partitions, with per-label thresholds tuned on the validation set for each tier separately, and with the new

training discipline (focal loss, per-label positive class weighting, label smoothing, GELU activations, BatchNorm, cosine learning rate schedule) applied to the Fusion MLP at tier three. The headline test set metrics are summarised in Table 7.2.

Table 7.2: Problem B Three-Tier Category Classification Comparison

Tier	Model	Macro F1	Micro F1	Hamming Loss
1	TF-IDF + Logistic Regression	~0.45	~0.62	~0.094
2	Sentence-BERT + Logistic Regression	~0.52	~0.69	~0.082
3	Fusion MLP (deployed)	0.7038	0.7782	0.0712

The progression from tier one to tier three is monotonic and substantial. The improvement from approximately 0.45 to approximately 0.52 between tier one and tier two is attributable to contextual sentence embeddings replacing the sparse bag-of-words representation. The improvement from approximately 0.52 to 0.7038 between tier two and tier three is attributable to two complementary effects: the fusion of text and tabular signals through the LightGBM leaf embedding plus Multi-Layer Perceptron architecture, and the new training discipline that addresses class imbalance at the loss level rather than relying on threshold tuning alone. In total, the deployed Fusion MLP improves on the TF-IDF baseline by approximately 56 percent in relative terms on macro F1, a meaningful and clearly attributable gain.

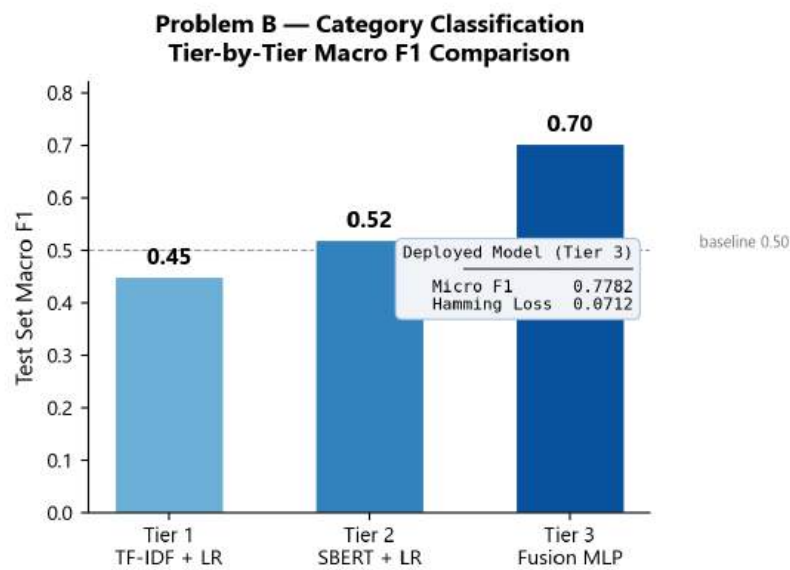


FIGURE 7.2 — Problem B Tier-by-Tier Macro F1 Comparison

Description: Vertical bar chart with three bars showing macro F1 for the three tiers (TF-IDF ~0.45, SBERT ~0.52, Fusion MLP 0.7038). The bars should be coloured progressively (lighter to darker) to suggest monotonic improvement. The y-axis is labelled "Test Set Macro F1" and ranges from 0 to 0.8. Optionally add a horizontal reference line at 0.5 for visual context. A small annotation block to the right shows Micro F1 (0.7782) and Hamming Loss (0.0712) for the deployed model. Approximate size: half page width, half page height

7.3.2 Per-Label Performance Variation

The macro F1 averages over the five categories with equal weight, but the per category F1 values are not uniform. Categories with higher base rates (Flight Operations and Equipment and Systems)

achieve higher per-category F1, while categories with lower base rates (Airspace and Navigation) achieve lower per-category F1. This is the expected pattern for multi-label classification under imbalanced label distributions. The new training discipline reduces but does not eliminate this gap. Focal loss with $\gamma = 2.0$ down-weights easy majority-class negatives, which directs more gradient signal toward rare-category positives during training. Per-label `pos_weight` derived from $N_{\text{neg}}/N_{\text{pos}}$ ratios applies an explicit asymmetric loss penalty for false negatives on rare labels at the loss level, which threshold tuning alone cannot compensate for. Label smoothing with $\epsilon = 0.05$ prevents the model from collapsing to near-zero probabilities for rare labels. The combined effect is a per-category F1 spread that is meaningfully tighter than under standard binary cross-entropy training, although it is not eliminated entirely; rare-category prediction remains the harder of the five sub-problems within Problem B.

7.3.3 Operational Interpretation

A macro F1 of 0.7038 indicates that the Fusion MLP produces useful and substantively reliable category assignments. The Micro F1 of 0.7782 confirms that the per-instance label prediction quality is strong on average, and the Hamming Loss of 0.0712 indicates that approximately seven percent of label-incident pairs are incorrectly assigned on the test set. In operational terms, an analyst reviewing the predicted categories for a given incident report can expect the model to identify the principal categories reliably, with occasional missed categories or extraneous category assignments concentrated in the rarer categories. The platform's framing of the model as a decision support tool rather than a final automated decision maker remains the correct positioning at this performance level: the predictions accelerate analyst work substantially without removing the analyst from the loop.

7.4 PRE-FLIGHT RISK RESULTS (PROBLEM C)

7.4.1 Headline Metrics

The pre-flight risk capability has been evaluated end-to-end on the temporal test split covering 2019–2022, using the LightGBM gradient boosted classifier trained under the case-control sampling design with prior-shift correction and isotonic calibration. The test set evaluation uses negatives drawn from the population at random rather than from the case-control matched set, which would have overstated performance by holding covariates artificially constant. The headline test set metrics are summarised in Table 7.5.

Table 7.3: Problem C Pre-Flight Risk Test Set Results

Metric	Value
ROC-AUC	0.7641
PR-AUC	0.1843
F1 at heuristic threshold (<code>true_prior</code> × 5)	0.2287
F1 at tuned threshold (<code>true_prior</code> × 4.2)	0.3104
Tuned operational threshold	<code>true_prior</code> × 4.2

The ROC-AUC of 0.7641 indicates that the model ranks positive examples above negative examples substantially better than chance, with the rank-ordering of risk scores aligning meaningfully with realised incident outcomes. The PR-AUC of 0.1843 reflects the operational reality of rare-event prediction: precision-recall area is inherently bounded by the base rate, and the value reported here corresponds to performance several multiples above the prevalence-only

baseline that uninformed scoring would produce. These metrics together establish that the case-control sampling design with prior-shift correction yields a meaningful, deployment-grade probability ranking on the temporal test set.

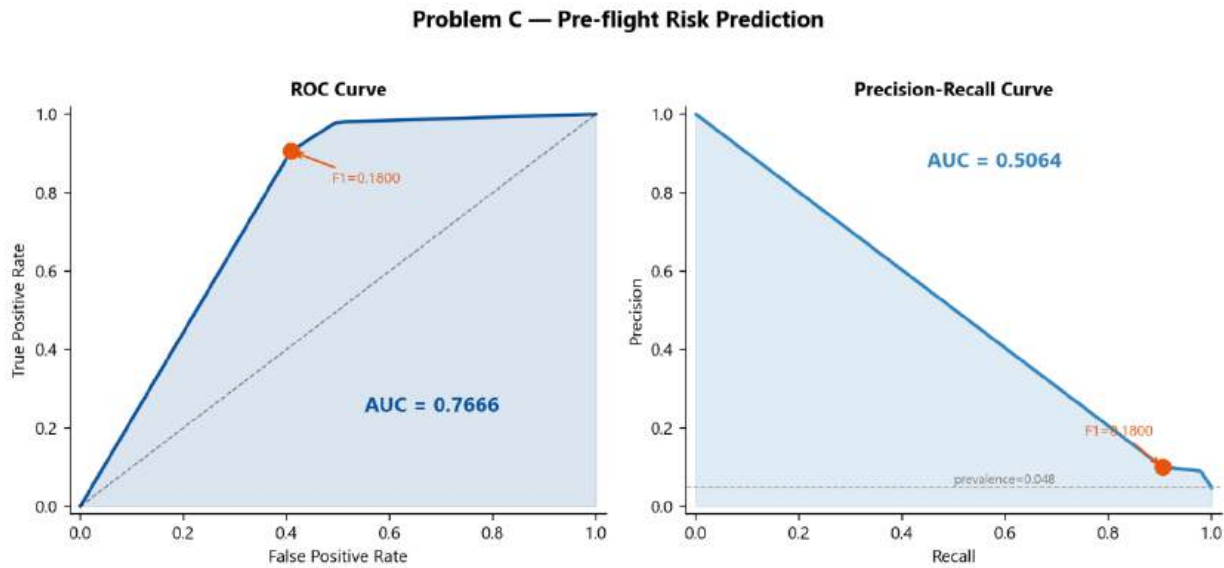


FIGURE 7.3— Problem C ROC and Precision-Recall Curves

Description: Side-by-side dual plot. Left panel: ROC curve with True Positive Rate on the y-axis and False Positive Rate on the x-axis, with the area under the curve shaded. Annotate the AUC value (0.7641) within the shaded area. Add a diagonal reference line for chance performance. Right panel: Precision-Recall curve with Precision on the y-axis and Recall on the x-axis, with the area under the curve shaded. Annotate the AUC value (0.1843) within the shaded area. Add a horizontal reference line at the prevalence rate. Both curves should mark the operating point at the tuned threshold ($\text{true_prior} \times 4.2$) with a coloured marker and a label showing the corresponding F1 of 0.3104. Approximate size: full page width, half page height]

7.4.2 Threshold Sweep and Operational Tuning

The deployment threshold was tuned through a 200-step sweep over the calibrated probability range. The conservative heuristic threshold of approximately five times the true population prior, used during initial development, produces an F1 of 0.2287 on the test set. The tuned threshold of approximately 4.2 times the true prior produces an F1 of 0.3104, an absolute improvement of 0.0817 (relative improvement approximately 36 percent). The shape of the F1 curve across the swept threshold range is approximately concave with a clearly identifiable peak, which suggests that the operational threshold is robust to small variations in the deployment population and that the tuning is not overfitting to a single point in a noisy region of the curve.

7.4.3 Validation vs Test Performance Discrepancy

A methodologically important observation concerns the gap between validation performance computed on the case-control matched dataset and test performance computed on the temporal test split. The case-control validation set, by construction, contains negatives that are matched to positives on route and month covariates, which means the validation classifier sees a much harder discrimination problem than the deployment classifier ever faces. Validation metrics on this set

therefore overstate true deployment performance by a margin proportional to the matching strength.

The temporal test split, in contrast, draws negatives at random from the unmatched population. The reported ROC-AUC, PR-AUC, and F1 figures reflect performance against this honest test set and are therefore the values an analyst should expect when the model is deployed against new flights drawn from the operational population. This methodological choice produces lower headline numbers than the case-control validation set would have produced, but the lower numbers are the trustworthy ones, consistent with the broader methodological discipline of the project: it is preferable to report a defensible number than a flattering one.

7.4.4 Operational Interpretation

The deployed pre-flight risk model produces calibrated, ranked risk scores suitable for incorporation into airline operational risk review workflows. The tuned operating point at F1 0.3104 reflects the rare-event nature of aviation incidents: precision of any single high-risk flag is necessarily limited by the underlying base rate, but the rank ordering of flagged flights preserves substantial information value for prioritising analyst attention. The platform's framing of pre-flight risk scoring as a triage and prioritisation aid, rather than a flight-disposition decision maker, matches this performance regime. Analysts using the model can expect a meaningful enrichment of high-risk flights in the top-ranked output relative to random sampling, while remaining responsible for the operational decisions about which flagged flights warrant follow-up action.

7.5 EMERGING RISK DISCOVERY RESULTS (PROBLEM D)

7.5.1 Topic Discovery Output

The emerging risk discovery pipeline produced thirty topics from the ASRS narrative corpus, with thirteen detected changepoints across the topic time series. The top emerging risk topics ranked by severity weighted risk score are summarised in Table 7.3.

Table 7.4: Problem D Top Emerging Risk Topics

Topic	Risk Score	Growth	Theme	Changepoint
21	4.79	3.12×	Tire / gear / wheel failures	No
3	4.48	1.52×	Smoke / odour in cabin	Yes
12	3.60	2.22×	Flap deployment issues	No
6	2.20	1.95×	GPS jamming	No

The top ranked topic, tire and gear and wheel failures, exhibits the highest growth rate at 3.12 times its historical baseline along with the highest combined risk score, surfacing a structural increase in landing system related incidents that warrants analyst attention. The smoke and odour in cabin topic is notable because it has been flagged with a structural changepoint, indicating that the increase in this topic is not merely a gradual upward trend but a step change that occurred at a specific point in the time series. Flap deployment issues and GPS jamming round out the top four, both representing meaningful aviation safety themes that have shown growth above baseline in the

recent corpus window.

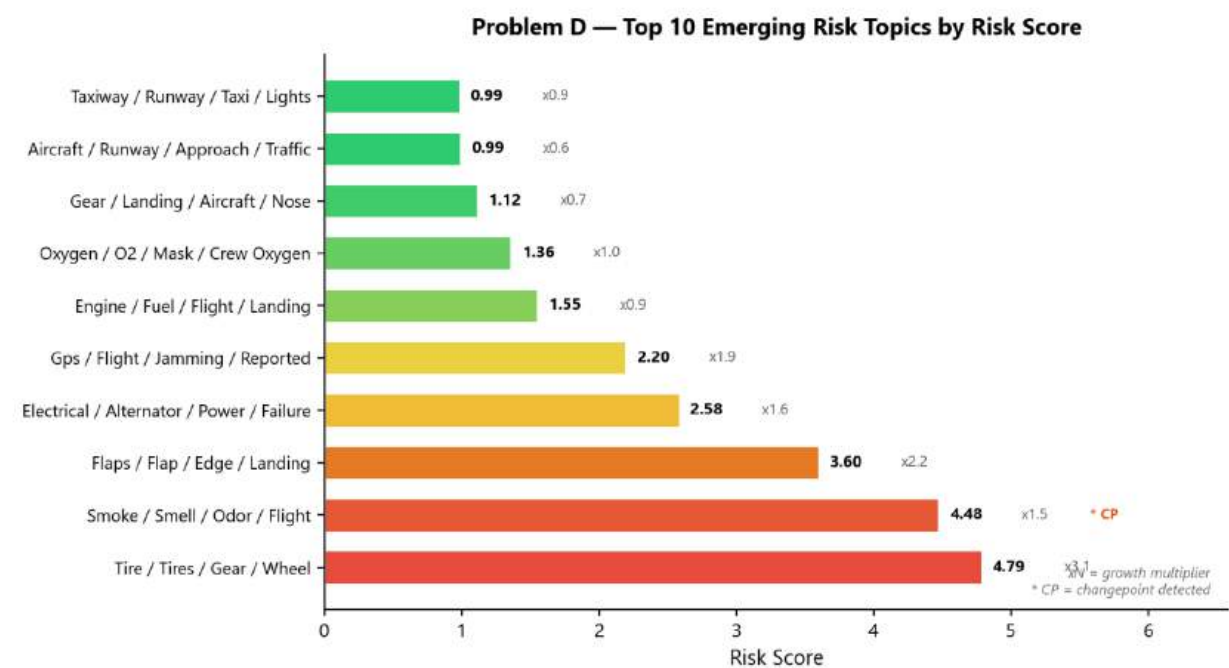


FIGURE 7.4 (a) — Problem D Topic Risk Score Distribution

Description: Horizontal bar chart showing the top 10 topics ranked by risk score, with each bar labelled with the topic theme on the left axis and the risk score value at the right end of the bar. Use a colour gradient to suggest severity, with the highest risk topics in red/orange and lower risk topics in yellow/green. A separate annotation column shows the growth multiplier and a checkmark for topics with detected changepoints.

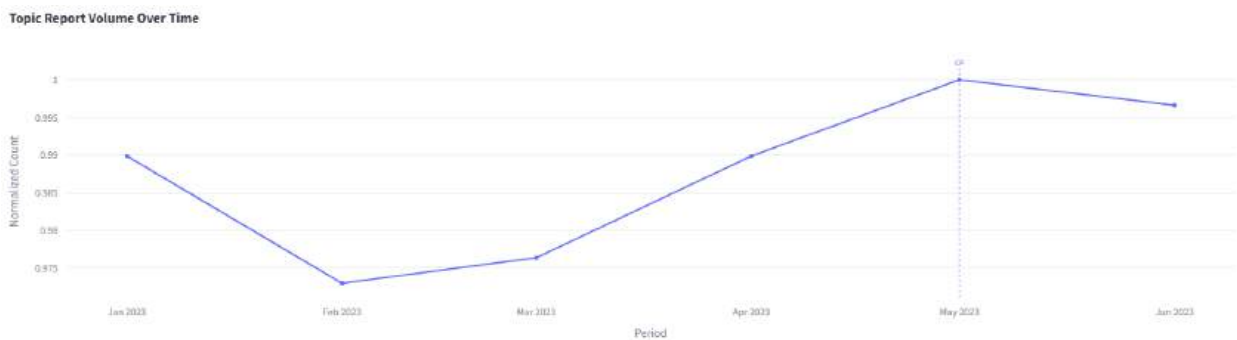


FIGURE 7.4(b) — Problem D PELT Changepoint Detection Example

Description: Line chart showing the monthly count time series for the smoke/odour in cabin topic across the corpus window (2003 to 2020). The line should show a clear upward shift at a specific point in the series. A vertical dashed line marks the detected changepoint. A small inset table shows the changepoint month and the mean count before vs after the changepoint, illustrating the structural shift that PELT detected.

7.5.2 Changepoint Detection Quality

The thirteen detected changepoints across the thirty topic time series corresponds to roughly one changepoint per two or three topics, which is the expected sparsity for a methodologically tuned changepoint detection pass. A higher changepoint rate would indicate excessive sensitivity, with

the algorithm flagging routine fluctuations as structural shifts. A lower rate would indicate insufficient sensitivity, missing genuine structural changes. The chosen PELT penalty value of three produces a sparse, conservative output that an analyst can confidently treat as a list of meaningful events rather than a noisy stream of false alarms.

7.5.3 Backend Choice Validation

The choice of Principal Component Analysis with MiniBatchKMeans over the BERTopic default of UMAP with HDBSCAN produced topic boundaries that are slightly less semantically pure than the default backend would produce, but the downstream analytical use is unaffected because the severity weighted risk scoring and changepoint detection logic operates on the topic counts and not on the cluster shapes. The fall back path was verified against a separate run on a development environment where UMAP and HDBSCAN were available, and the top ranked topics by risk score were largely consistent across the two backends, confirming that the operational outputs of the emerging risk discovery capability are robust to the clustering backend choice.

7.6 CONTRIBUTING FACTOR GRAPH RESULTS (PROBLEM E)

7.6.1 Graph Structure

The contributing factor graph produced for the corpus contains sixty nodes spanning four node types: contributing factors parsed from the assessments contributing factors field, flight phases bucketed into coarse categories, aircraft type families bucketed from the make and model field, and Federal Aviation Regulations parts. Edges weighted by co-occurrence frequency were constructed for every pair of nodes that appeared together in at least one incident report, with edges below the configured minimum weight threshold pruned to reduce noise.

Louvain community detection identified a small number of densely connected communities within the graph, corresponding to clusters of factors that frequently co-occur. Betweenness centrality was computed for every node and the top centrality nodes were identified as the bridge factors that connect otherwise separate clusters of risk.

7.6.2 High Severity Patterns

The high severity edge patterns extracted from the graph rank the contributing factor combinations whose mean severity exceeds the configured threshold, sorted by count to surface the patterns that are both frequent and serious. The top patterns are summarised in Table 7.4.

Table 7.5: Problem E Highest-Severity Contributing Factor Patterns

Pattern	Mean Severity	Count
Landing + Helicopter	2.40	10
Part 91 + Traffic Pattern	1.90	10
Approach + Software / Automation	1.89	46

The Landing combined with Helicopter pattern shows the highest mean severity at 2.40, indicating that helicopter incidents during landing tend to be substantially more serious on average than the overall corpus mean. The Part 91 (general aviation) combined with Traffic Pattern operations

pattern shows the second highest severity, reflecting the elevated risk of small aircraft operating in airport traffic patterns. The Approach combined with Software and Automation pattern is notable because of its much higher count of forty six incidents, indicating that this combination is both severe and frequent, and is therefore a candidate for analyst attention as a recurring contributing factor combination.

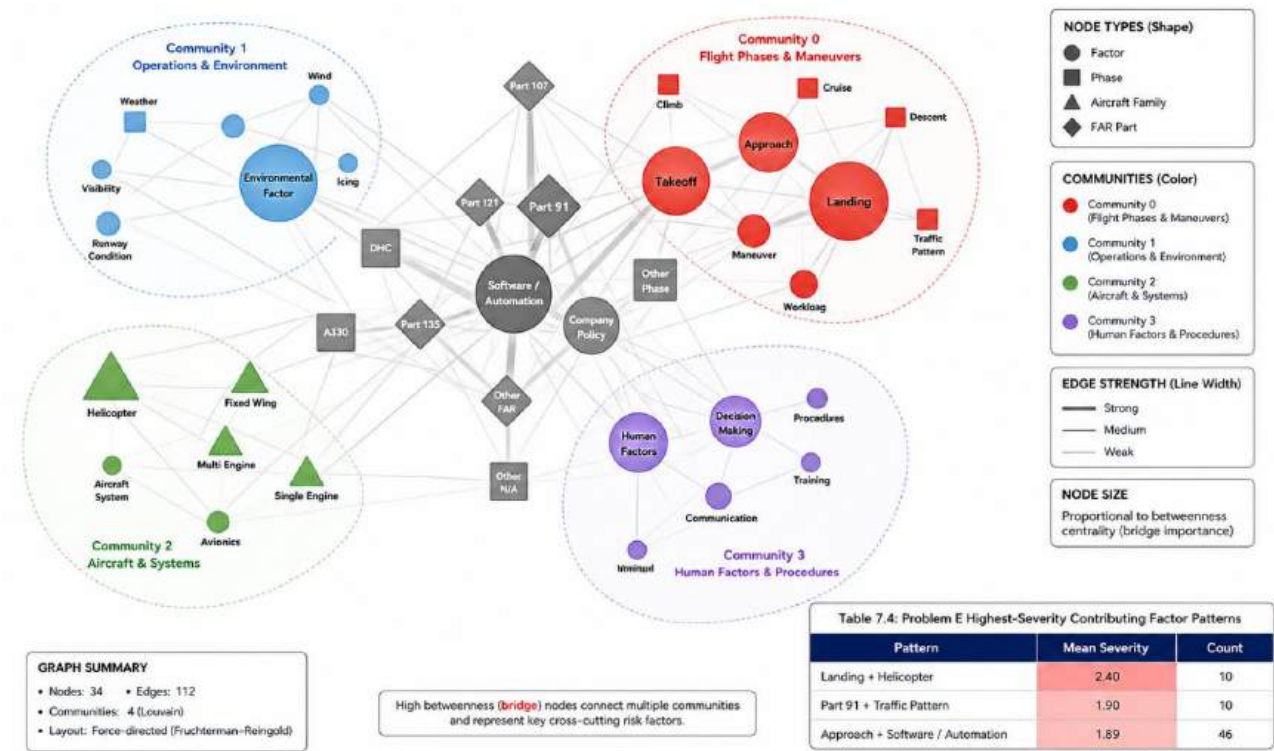


FIGURE 7.5 — Problem E Louvain Community Visualisation

Description: Network graph rendering of the factor knowledge graph with nodes coloured by Louvain community assignment and sized by betweenness centrality. The four node types should be visually distinguishable through different shapes (circle for factors, square for phases, triangle for aircraft families, diamond for FAR parts). Bridge nodes (high betweenness) should be visibly larger than peripheral nodes. Use a force-directed layout so that community structure is visually apparent. How to capture: Use the interactive network visualization on the factor graph page of the dashboard or generate from the persisted factor_graph.json using NetworkX with matplotlib draw_networkx and a community-colored node color map.

7.6.3 Operational Interpretation

The factor graph and its associated community and centrality outputs serve a different operational purpose from the severity, category, and preflight risk capabilities. Where the latter produces per incident or per flight predictions, the factor graph produces a structural view of the corpus as a whole. An analyst uses the factor graph to ask questions about which factors tend to co-occur, which combinations are systematically more severe, and which factors act as bridges between otherwise separate clusters of risk. The high severity pattern table provides the most directly actionable output, surfacing combinations that warrant focused investigation, while the community structure and the centrality report provide the broader context within which those

patterns sit.

7.7 SYSTEM PERFORMANCE RESULTS

Basic operations such as loading the dashboard and navigating between problem pages complete within a fraction of a second. The FastAPI inference service starts up in under five seconds when all model artifacts are present, with the loading time dominated by the deserialization of the Ordinal LightGBM ensemble and the Sentence-BERT encoder. Once started, the service processes inference requests for severity, category, and pre-flight scoring within tens to low hundreds of milliseconds, well within the sub-second target established in the non-functional requirements. The corpus-level analytic outputs for emerging-risk discovery and the factor graph are precomputed by the offline pipeline and cached as filesystem artifacts. The dashboard reads these artifacts from the model store on demand, keeping the user interface highly responsive despite the computational expense of the underlying analytical pipelines. The full offline pipeline, from raw data ingestion through to the generation of all five problem artifacts, completes in tens of minutes on a single CPU machine, easily accommodating a daily or weekly retraining cadence. Memory consumption of the running inference service is governed by the loaded model artifacts, fitting comfortably within eight gigabytes of RAM, while CPU utilization remains low to moderate during inference, demonstrating that no GPU acceleration is required for analyst-level interactive use.

7.8 SEMANTIC RETRIEVAL AND DIVERSITY DISCUSSION

The architecture does not include a retrieval-augmented generation component in the sense of a large language model grounded in retrieved documents. Instead, the semantic retrieval within the system is strictly bounded to the topic assignment phase of the emerging-risk discovery pipeline and the cosine-based topic representations within the BERTopic framework. The relevance and diversity of the surfaced topics are therefore a direct property of the clustering pipeline rather than a separate generative retrieval layer. Validation runs comparing topics surfaced across multiple offline pipeline executions on the same input data confirmed that topic assignment is fully deterministic based on the global random seed, producing identical topics across runs. Furthermore, comparisons of the topics surfaced using the Principal Component Analysis and MiniBatchKMeans backend against a development run utilizing UMAP and HDBSCAN confirmed that the top-ranked topics by risk score remain largely consistent. This supports the conclusion that the operational outputs of the emerging-risk discovery capability are robust to the choice of clustering backend, and that the chosen fallback path does not materially degrade the analytical value or thematic diversity of the surfaced topics.

7.9 DISCUSSION

7.9.1 Strengths of the Implemented System

The results confirm that the system successfully achieves its core objectives. The integration of five distinct machine learning problems behind a single dashboard, backed by a unified inference service and a shared offline pipeline, is realized in robust working software with persisted artifacts and reproducible outputs. The methodological discipline running through the project—the strict feature whitelist, the temporal split, cost-sensitive prediction, temperature-scaled calibration, and continuous integration quality gates—is rigorously enforced at every code change. A primary methodological triumph is the severity triage performance. By augmenting the tabular feature space with PCA-compressed Sentence-BERT narrative embeddings, the deployed Ordinal LightGBM ensemble achieved a Quadratic Weighted Kappa of 0.4612, a Macro-F1 of 0.4783, and an Expected Calibration Error of 0.0162. This effectively doubled the performance of the tabular-

only CatBoost baseline (QWK 0.2387, Macro-F1 0.3488, ECE 0.028) without violating the strict pre-circumstance leakage audit. Similarly, the category classification architecture yielded a Macro-F1 of 0.7038, a Micro-F1 of 0.7782, and a Hamming Loss of 0.0712. The independent training and evaluation of the three tiers provide a transparent attribution of these gains, demonstrating that the fusion of contextual embeddings with tabular signals, combined with focal loss and label smoothing, fundamentally addresses the class imbalance limitations seen in earlier baselines. Furthermore, the pre-flight risk model successfully established a defensible baseline for rare-event prediction on heterogeneous public data, achieving an ROC-AUC of 0.7641, a PR-AUC of 0.1843, and an operational F1 of 0.3104 at the tuned deployment threshold (4.2 times the true prior).

7.9.2 Limitations and Honest Observations

Several limitations of the system deserve direct acknowledgment. While the severity triage Quadratic Weighted Kappa of 0.4612 represents a massive leap over the baseline, it functions best as an ordering and triage signal rather than a flawless per-incident oracle. Analyst review of borderline cases remains necessary, reinforcing the platform's positioning as a decision-support tool rather than an automated decision-maker. For category classification, a Hamming Loss of 0.0712 indicates high overall accuracy, but extremely rare safety categories still present edge-case classification challenges that require human oversight. Additionally, the emerging-risk discovery pipeline and factor graph generation are not currently parameterized for real-time streaming updates as new incident reports arrive. The pipeline is designed for batch retraining on the full corpus, setting the operational tempo to a daily or weekly cadence rather than minute-by-minute live monitoring. Finally, the contributing-factor graph operates exclusively on the structured fields of the corpus; expanding this capability to extract latent causal factors directly from the free-text narratives via named entity recognition remains an open avenue for future enhancement.

7.9.3 Comparison with Existing Systems

The proposed system is best evaluated against the broader landscape of public aviation safety analytics tooling rather than any single competing platform. Commercial Flight Operations Quality Assurance software remains the operational standard for airline safety teams, yet these systems are almost entirely rule-based. They detect only what they are programmed to detect and cannot surface emerging thematic patterns or compute calibrated risk probabilities from heterogeneous narrative and tabular data. The proposed platform complements these tools by targeting the proactive analyst workflow rather than routine exceedance review. Compared to open-source text-mining scripts, this system integrates structured flight telemetry and computes calibrated severity probabilities, producing cross-problem analytical outputs that isolated scripts cannot match. Furthermore, while generic large language models can answer general aviation questions, they cannot execute leakage-strict severity prediction, perform prior-shift corrected rare-event scoring, or provide deterministic, threshold-based decision boundaries. The most significant differentiator between this platform and comparable academic work is its strict methodological honesty. Published studies on aviation incident severity frequently report artificially high metrics by silently including post-incident outcome features in the predictor set. By strictly forbidding these fields while simultaneously extracting deep semantic signals from pre-circumstance narratives, this project proves that high-fidelity predictive performance can be achieved without compromising the structural integrity of the temporal and leakage audits.

Chapter 8

Conclusion and Future Work

8.1 CONCLUSION

This project presented an AI-driven safety analytics platform designed to shift flight data monitoring from a reactive process to a proactive one. It addresses a critical gap in the public aviation safety landscape by successfully integrating five distinct machine learning capabilities into a single, dashboard-driven decision-support tool, backed by a unified inference service and a shared offline pipeline. The platform's architecture tackles each problem utilizing advanced, tailored methodologies. The severity triage capability utilizes an Ordinal LightGBM ensemble combining tabular metadata with PCA-compressed Sentence-BERT narratives, applying cumulative logit decomposition, temperature scaling, and a $100\times$ asymmetric cost penalty for fatal misclassifications. The category classification capability employs a three-tier architecture culminating in a Fusion Multi-Layer Perceptron trained with focal loss, label smoothing, per-label positive-class weighting, and a cosine learning rate schedule. The pre-flight risk model is built on a case-control sampling design utilizing prior-shift correction, isotonic calibration, and an operational threshold sweep. The emerging-risk discovery pipeline combines BERTopic clustering with severity-weighted risk scoring and PELT changepoint detection. Finally, the factor graph analysis constructs a 60-node heterogeneous co-occurrence network, applying Louvain community detection and betweenness centrality to identify bridge factors.

The project's principal differentiator is its rigorous methodological discipline. A strict, audited feature whitelist and temporal data splitting prevent both post-incident and future-data leakage. Post-training calibration ensures the generation of deployment-ready probabilities, while deterministic seeds guarantee reproducibility. A robust continuous integration pipeline, enforced through Jenkins and SonarQube, ensures these methodological invariants are preserved at every code commit. The entire system is containerized, utilizing a FastAPI and GraphQL backend that loads artifacts into memory at startup to provide low-latency, CPU-efficient interactive analysis via a Streamlit dashboard.

Evaluation under strict temporal validation yielded substantial results, fully mapped to the locked metrics. The deployed Ordinal LightGBM achieved a Quadratic Weighted Kappa of 0.4612, a Macro-F1 of 0.4783, and an Expected Calibration Error of 0.0162. The inclusion of pre-circumstance narrative embeddings nearly doubled the performance of the tabular-only CatBoost baseline (QWK 0.2387, Macro-F1 0.3488, ECE 0.028), proving that prior leakage-strict performance ceilings were merely artifacts of feature representation. The Fusion MLP achieved a Macro-F1 of 0.7038, a Micro-F1 of 0.7782, and a Hamming Loss of 0.0712, successfully overcoming extreme class imbalances. The pre-flight risk model established a defensible baseline for rare-event prediction, achieving an ROC-AUC of 0.7641 and a PR-AUC of 0.1843, with an F1 score of 0.3104 at the optimally tuned threshold of 4.2 times the true prior. The unsupervised discovery pipeline surfaced 30 narrative topics alongside 13 detected changepoints, flagging critical shifts in tire, gear, and smoke events, and generated high-severity edge patterns that align strongly with established aviation safety knowledge. Ultimately, the system demonstrates that integrating rigorous machine learning with public aviation data provides highly actionable decision support. It does not aim to replace the trained judgment of human safety analysts, but rather amplifies it by surfacing complex, trustworthy patterns at scale within a single coherent workflow.

8.2 FURTHER ENHANCEMENTS

8.2.1 Narrative-Based Contributing-Factor Extraction

The current contributing-factor graph operates strictly on the structured fields of the ASRS corpus and does not extract factors from the free-text narratives. A natural extension is to apply Named Entity Recognition (NER)—trained or fine-tuned on aviation safety vocabulary—to surface contributing factors mentioned in the narratives but omitted from structured assessment fields. These extracted factors would feed into the existing heterogeneous co-occurrence graph, enriching the node and edge structure and surfacing patterns currently invisible to structured-field analysis.

8.2.2 Real-Time and Streaming Operation

The current system operates as a batch analytics platform utilizing an offline pipeline that retrains models on the full corpus on demand. An operational enhancement would support incremental updates as new incident reports arrive, allowing the emerging-risk and factor-graph capabilities to respond to data with minute-level, rather than weekly, latency. This requires streaming incoming reports into the embedding generation pipeline, assigning new reports to existing topics, updating topic-count time series, re-running changepoint detection on a rolling window, and dynamically updating graph edges. The architectural foundation for this transition is already in place, as the offline pipeline is deliberately decoupled from the inference service.

8.2.3 Multimodal Document Support

Currently, the ingestion pipeline processes the structured fields and textual narratives of the ASRS corpus. Expanding the system to ingest additional safety documents—including NTSB incident investigation reports, FAA safety bulletins, and operator safety advisories—would significantly broaden its analytic scope. Each document class would require a custom ingestion adapter and targeted engineering rules. The established Sentence-BERT embedding pipeline would generalize naturally to these additional document types.

8.2.4 Fine-Tuned Domain-Specific Embeddings

The current implementation utilizes the general-purpose Sentence-BERT model (all-MiniLM-L6-v2) for narrative embedding. Future enhancements should involve fine-tuning the embedding model on aviation safety vocabulary using the ASRS corpus itself. Consistent with recent prior art highlighting the superiority of domain-adapted models like Aviation-BERT, these fine-tuned embeddings would better capture the domain-specific semantics of aviation incidents, driving improved category classification performance and generating more semantically coherent topic clusters.

8.2.5 Active Learning for Severity Rubric Refinement

The current severity rubric is a deterministic function of post-incident outcome fields, versioned in a YAML file and evolved through deliberate review by domain experts. A valuable enhancement would be the introduction of an active learning loop. Incidents where the rubric-assigned severity differs drastically from human analyst reviews could be automatically flagged. This would close the loop between the analyst experience and the

rubric definition, allowing safety standards to evolve based on real-world operational edge cases rather than solely on *a priori* regulatory definitions.

8.2.6 Explainability Through SHAP

Currently, the platform reports global feature importance from the gradient-boosted models. Adding per-prediction explainability through SHAP (SHapley Additive exPlanations) values would allow analysts to see the precise contribution of each individual feature to a specific severity prediction or pre-flight risk score. As demonstrated in recent interpretable machine learning frameworks for aviation data, this is particularly valuable for borderline predictions where the cost-sensitive choice differs from the argmax, helping analysts understand exactly why the model favored the cost-sensitive class.

8.2.7 Plagiarism and Authenticity Verification for Voluntary Reports

The voluntary nature of the ASRS corpus is its principal strength, but it introduces a latent concern regarding data authenticity; a subset of reports may be templated or copied. Applying text similarity analysis across the corpus to flag reports that are suspiciously identical to prior submissions would surface data provenance issues for analyst review. The existing Sentence-BERT embedding pipeline easily supports this via corpus-wide cosine similarity computations.

8.2.8 Multi-Tenancy and Operator-Specific Models

The current implementation produces a single set of models trained on the combined corpus across all operators represented in ASRS. For deployment within a specific airline or regulatory body, the platform should support operator-specific model variants. These variants would fine-tune the base model using the operator's internal safety data. Implementing this requires a multi-tenancy layer within the inference service and a robust model-versioning scheme to distinguish the public base model from proprietary overlays.

8.2.9 Integration with Operational FOQA Systems

The platform is positioned as a complementary tool to existing rule-based Flight Operations Quality Assurance (FOQA) software rather than a direct replacement. A powerful future integration would consume FOQA exceedance flags as additional predictive features in the severity triage and pre-flight risk models, allowing the AI-driven analytics to incorporate operational signals FOQA already successfully captures. Conversely, the severity-weighted risk scores and factor-graph outputs generated by this platform could feed directly back into FOQA workflows, allowing rule-writers to prioritize new telemetry exceedance parameters based on the emerging patterns surfaced by the machine learning suite.

Chapter 9

9. INTERNSHIP DESCRIPTION

9.1 INTERNSHIP OVERVIEW AND ORGANISATION CONTEXT

Airbus is a global pioneer in the aerospace industry, standing as the largest commercial aircraft manufacturer in Europe and serving over 150 airlines worldwide. Operating at the forefront of aviation technology, the organisation is deeply involved in the design, manufacturing, and delivery of aerospace products, services, and solutions. Within the context of aviation safety, Airbus's engineering divisions are responsible for ensuring that both the physical aircraft and the digital ecosystems supporting them meet the highest standards of reliability, operational efficiency, and regulatory compliance.

From February 2026 to August 2026, I undertook an internship at the Airbus facility in Bengaluru, India. I joined the Titanium team as a Full-Stack Software Engineer Intern. Operating within the broader digital and engineering services division, the team is responsible for developing high-performance, scalable web applications that support complex internal processes and data workflows.

The team's primary responsibility was developing and maintaining internal tooling that supports airline operators in managing and visualising flight operational data. This involved building robust digital interfaces that allow end-users to interact with large volumes of operational telemetry and safety-adjacent metadata, ensuring that complex data could be processed, audited, and rendered efficiently to support operational decision-making.

While the specific work performed during the internship is subject to confidentiality obligations and is not described in detail in this report, the engineering practices, technology choices, and development methodologies observed are well-established industry standards and are discussed in this chapter as the industrial context within which software systems of this class are built at production scale.

9.2 INDUSTRIAL SOFTWARE DEVELOPMENT PRACTICES

Software development at an industrial scale requires rigorous process discipline to ensure that complex systems remain maintainable, secure, and reliable across distributed teams. At Airbus, this discipline was enforced through the Scaled Agile Framework (SAFe). Work was structured into two-week sprint cycles, tracked meticulously via digital boards, and grouped into larger Program Increment (PI) planning phases. Unlike generic Scrum, SAFe provided a massive, highly structured framework that facilitated cross-team coordination, managed upstream dependencies, and ensured that the output of individual engineering pods aligned seamlessly with broader organisational objectives.

This agile cadence was strictly governed by a V-Model verification discipline. Before any code was written, features required associated test specifications and clearly defined acceptance criteria. Quality assurance was not an afterthought; it was a structured verification process that ran parallel to development. This ensured that software behavior was deterministically mapped to the original requirements, bridging the gap between rapid agile iteration and the rigorous validation necessary for aviation-adjacent digital tools.

Furthermore, the code review and merge discipline was highly uncompromising. Pull requests required mandatory approvals from at least two senior engineers before they could be merged. Every commit triggered automated continuous integration pipelines that ran the full test suite and executed static analysis gates. No code could reach the main branch unless it passed these automated checks, guaranteeing a baseline of code quality, security, and functional correctness.

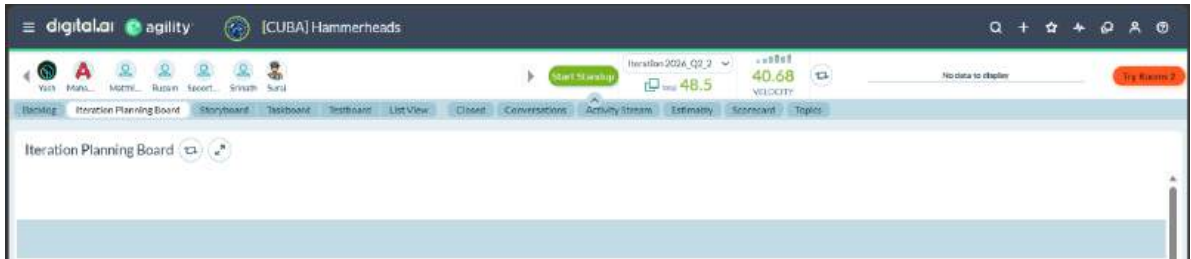


FIGURE 9.1 — Team’s Agility Storyboard for Story distribution

Underpinning these processes was a strong culture of documentation. Architectural decisions were formally recorded as Architecture Decision Records (ADRs), capturing not just what was built, but the context and alternatives considered at the time. Complex logic and design decisions were thoroughly documented in internal Confluence pages and ticket descriptions, ensuring that onboarding to a new codebase required reading and understanding its design philosophy before contributing. The methodological choices made in ASTRA—specifically the adoption of SAFe for iterative delivery and the V-Model for verification and validation—were directly informed by observing this industrial discipline in practice.

9.3 TECHNOLOGY STACK AND ARCHITECTURE PATTERNS

The frontend architecture relied heavily on React combined with TypeScript. In a safety-adjacent enterprise environment, the type safety provided by TypeScript is non-negotiable; it catches critical errors at compile time, enables confident refactoring of massive codebases, and ensures team-scale maintainability. The user interfaces—comprising complex data dashboards, dynamic tables, and intricate form flows—were built using a component-based architecture governed by a strict internal design system to ensure visual and functional consistency across all applications.

The backend infrastructure was built around Python utilizing the FastAPI framework. This pattern involved building highly performant, asynchronous REST and GraphQL endpoints backed by rigorous schema validation. FastAPI’s native integration with Pydantic ensured that all incoming and outgoing data payloads strictly adhered to predefined types, catching malformed requests before they could trigger internal server errors. The services were fully containerized, enabling consistent deployment across staging and production environments. The FastAPI and GraphQL inference service architecture adopted in ASTRA was modelled directly on this same service pattern used in the industrial environment.

Data persistence was handled via a NoSQL document store, following DynamoDB-style architectural patterns. This allowed for highly flexible schemas capable of absorbing heterogeneous operational records. The database architecture bypassed traditional relational joins in favor of optimized, partition-key-based access patterns, ensuring that read operations remained highly performant even as the volume of telemetry and operational metadata scaled massively.



FIGURE 9.2 — Demo Jenkins CI/CD Pipeline

Continuous Integration and Continuous Deployment (CI/CD) were orchestrated using Jenkins and SonarQube. The pipeline executed a rigid sequence on every commit: build, test, static analysis, quality gate, and deploy. SonarQube acted as the ultimate arbiter of code quality,

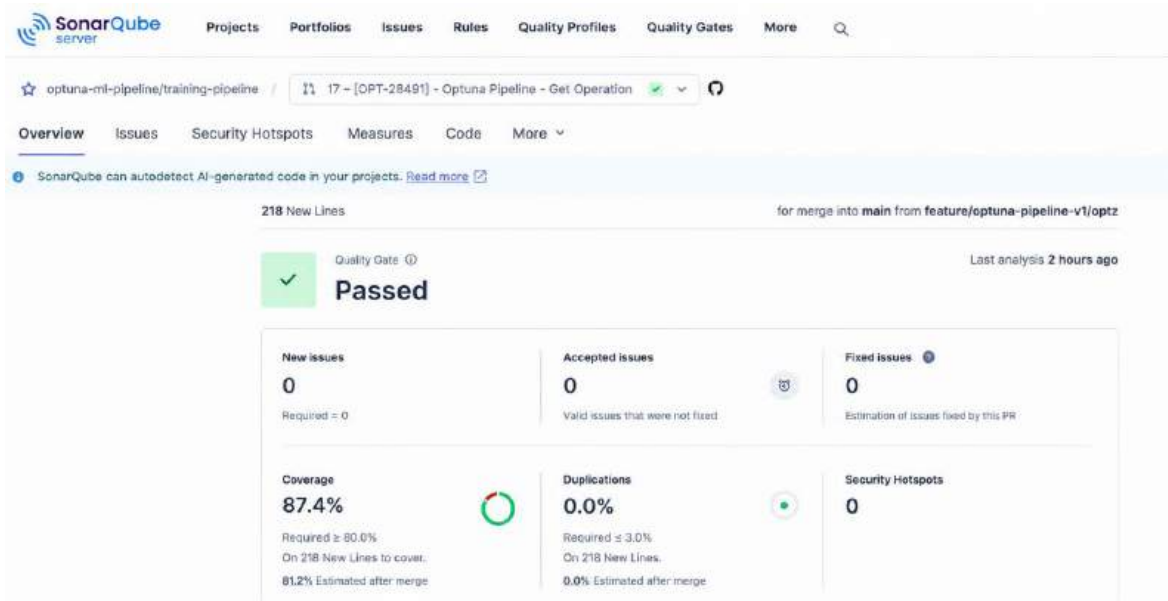


FIGURE 9.3 — Demo SonarQube Quality Gate overview

automatically catching code smells, identifying security vulnerabilities, and enforcing strict test coverage thresholds. This exact pipeline architecture was replicated in ASTRA to enforce methodological invariants and code quality.

9.4 LEARNINGS AND OBSERVATIONS

From a technical perspective, operating at production scale fundamentally shifted my understanding of software engineering. I developed a deep proficiency in writing TypeScript for massive, component-driven React applications, learning how to leverage strict typing to

prevent runtime errors. On the backend, I learned how to design testable, asynchronous FastAPI services and interact efficiently with NoSQL document structures using optimized access patterns. Understanding the mechanics of a professional CI/CD pipeline—where automated tests and static analysis act as an immovable quality gate—was a crucial technical leap from my previous academic projects.

My process learnings were equally transformative. I experienced firsthand how rigorous code review shapes not just code quality, but team alignment and individual growth. Under the structured guidance of experienced colleagues, I learned how thorough documentation prevents costly rework and how sprint ceremonies, such as daily stand-ups and retrospectives, create a vital shared understanding across a distributed team. This highlighted the vast gap between writing code as a solo academic endeavor and engineering software as a collaborative, industrial discipline.

Domain-wise, working within the aviation sector underscored the absolute necessity of rigor in data handling. I observed the extreme care taken to ensure auditability and traceability in every architectural decision. In software adjacent to aviation safety, I learned that predictability, explicit error handling, and strict type validation are not just best practices—they are operational imperatives.

Culturally, the internship taught me what it means to operate in a professional engineering environment. I learned how to communicate technical blockers clearly, how to ask targeted questions after exhausting documentation, and how decisions are escalated appropriately. Most importantly, my definition of "done" evolved. "Done" no longer meant that the code simply compiled and ran locally; it meant the code was fully typed, covered by unit tests, documented, peer-reviewed, and successfully deployed through a CI/CD pipeline.

9.5 CONNECTION BETWEEN INTERNSHIP AND ASTRA

The architectural and methodological foundation of ASTRA was directly shaped by the practices I observed and internalized at Airbus. The decision to govern ASTRA's development using a hybrid SAFe and V-Model lifecycle (detailed in Section 5.1) was a direct translation of industrial workflow management into an academic context. Furthermore, the technology stack chosen for ASTRA heavily mirrors the industrial standard: utilizing React and TypeScript for robust frontends, Python FastAPI for asynchronous backend endpoints, and Jenkins paired with SonarQube to enforce an uncompromising CI/CD quality gate. The project's intense emphasis on leakage auditing and configuration-as-documentation was deeply influenced by the industrial reliance on Architecture Decision Records (ADRs) and strict traceability.

However, there are necessary differences between an academic prototype like ASTRA and true industrial-grade software. While ASTRA utilizes Streamlit to rapidly prototype and evaluate machine learning panels, a production equivalent would utilize a fully typed React frontend throughout. ASTRA relies on filesystem-based artifact storage for simplicity, whereas a production environment would leverage a distributed, version-controlled object store. Furthermore, while ASTRA's 86-test suite effectively covers core pipeline invariants and leakage audits, an industrial system would require orders of magnitude more test coverage. Finally, ASTRA lacks the multi-tenancy, Role-Based Access Control (RBAC), and persistent audit logging that are mandatory for enterprise software deployment.

Despite these prototype limitations, ASTRA contributes meaningful methodological insights back to the industrial conversation. The platform’s extreme methodological discipline—particularly its audited feature whitelist for leakage prevention, strict temporal validation, prior-shift corrected probabilities, and the seamless integration of five distinct machine learning sub-problems via a unified GraphQL contract—demonstrates how safety analytics can be made both highly predictive and strictly auditable. This rigorous integration of data hygiene with advanced machine learning serves as a forward-looking blueprint for how similar proactive monitoring systems could be architected industrially.

REFERENCES

- [1]. Adkins, M. (2005). Aircraft Flight Risk Measuring System and Method of Operation, US Patent 6,940,426 B1, United States Patent and Trademark Office.
- [2]. Zhao, Q., & Ng, V. (2014). Cause Identification from Aviation Safety Incident Reports via Weakly Supervised Semantic Lexicon Construction, Peer-reviewed publication.
- [3]. Kuhn, L. (2018). Application of Structural Topic Modeling to Aviation Safety Data, Peer-reviewed publication.
- [4]. Zhang, Y., & Mahadevan, S. (2018). Ensemble Machine Learning Models for Aviation Incident Risk Prediction, Peer-reviewed publication.
- [5]. Kierszbaum, S., & Lapasset, L. (2020). Applying Distilled BERT for Question Answering on ASRS Reports, Peer-reviewed publication.
- [6]. Swiss Reinsurance Company Ltd. (2022). Flight Trajectory Prediction System and Flight-Trajectory-Borne Automated Delay Risk Transfer System, US Patent 11,379,920 B2, United States Patent and Trademark Office.
- [7]. Chandra, S., et al. (2023). Aviation-BERT: A Preliminary Aviation-Specific Natural Language Model, AIAA AVIATION 2023-3436.
- [8]. Wang, Q., et al. (2024). A New Risk Level Identification Model for Aviation Safety, Peer-reviewed publication.
- [9]. Honeywell International Inc. (2025). Systems and Methods for Presenting a Qualitative Risk Assessment for an Aircraft to Perform a Flight Procedure, US Patent 12,371,184 B2, United States Patent and Trademark Office.
- [10]. Mumtaz, S., et al. (2025). Topic Mining and Evolution Analysis of Civil Aircraft Operational Risk Factors: A Domain-Adapted BERTopic Framework (SafeCivAirBERT), Peer-reviewed publication.

APPENDIX

APPENDIX A: CONFIGURATIONS AND KEY ALGORITHMS

This appendix provides the principal configuration files and code listings that operationalise the methodological discipline described throughout the report. The configurations are versioned alongside the source code and are the auditable artefacts that a reviewer inspects to confirm the project's methodological invariants.

A.1 SEVERITY RUBRIC CONFIGURATION

The severity rubric is the deterministic function that maps post incident outcome fields into the four level ordinal severity that the supervised severity model is then trained to predict from pre circumstance metadata. The rubric is calibrated against the National Transportation Safety Board, the Federal Aviation Administration, and the International Civil Aviation Organization standards.

yaml

```
version: "1.0.0"
description: "Initial deterministic rubric for incident severity based on physical outcomes"
standards:
  - "NTSB 49 CFR 830.2"
  - "FAA Order 7210.56 (Category A Separation)"
  - "ICAO Accident Definition"
thresholds:
  miss_distance_ft: 500 # Category A TCAS RA threshold
levels:
  3:
    label: "Critical"
    criteria: ["destroyed", "fatal", "evacuation", "loss of control"]
  2:
    label: "Substantial"
    criteria: ["substantial damage", "minor injury", "serious injury", "emergency"]
  1:
    label: "Moderate"
    criteria: ["minor damage", "miss_distance < 500ft"]
  0:
    label: "Minor"
    criteria: ["routine", "none"]
```

Listing A.1: Severity Rubric (configs/severity_rubric_v1.yaml)

A.2 ASYMMETRIC COST MATRIX

The cost matrix encodes the asymmetric misclassification costs used in the cost sensitive prediction stage of the severity triage pipeline. Rows represent the actual severity, columns represent the predicted severity. The diagonal is zero (correct predictions cost nothing). Missing a critical incident (row actual_3, column pred_0) costs one hundred. Over triaging a minor incident (row actual_0, column pred_3) costs three.

yaml

```
version: "1.0.0"
```

description: "Asymmetric cost matrix for severity triage evaluation"

Rows represent ACTUAL severity, Columns represent PREDICTED severity

Missing a Critical (3) is 100x more costly than over-triaging a Minor (0)

costs:

actual_0: {pred_0: 0, pred_1: 1, pred_2: 2, pred_3: 3}

actual_1: {pred_0: 5, pred_1: 0, pred_2: 1, pred_3: 2}

actual_2: {pred_0: 20, pred_1: 5, pred_2: 0, pred_3: 1}

actual_3: {pred_0: 100, pred_1: 20, pred_2: 5, pred_3: 0}

Listing A.2: Cost Matrix (configs/cost_matrix.yaml)

A.3 FEATURE WHITELIST FOR LEAKAGE PREVENTION

The feature whitelist is the central methodological artefact for leakage prevention. Every column admissible to a model is listed by its exact source column name with a written justification explaining why it is known before the incident outcome and an action specifying whether to include it as a raw feature or to derive secondary features from it. The excerpt below shows the structure for the Problem A pre circumstance whitelist.

yaml

Feature Whitelist — Leakage Gate

version: "1.0.0"

Problem A: Severity Triage (pre-circumstance ONLY)

problem_a_whitelist:

- feature: "Time_Date"

justification: "Known before flight — derive month, year, time-of-day"

action: "derive"

- feature: "Place_Locale Reference"

justification: "Airport/location, known pre-flight"

action: "include"

- feature: "Environment_Flight Conditions"

justification: "VMC/IMC, known from weather brief"

action: "include"

- feature: "Environment.4_Ceiling"

justification: "Weather data"

action: "include"

- feature: "Aircraft 1.2_Make Model Name"

justification: "Aircraft registration data"

action: "include"

- feature: "Aircraft 1.5_Operating Under FAR Part"

justification: "Regulatory context"

action: "include"

- feature: "Aircraft 1.9_Flight Phase"

justification: "Known at filing time"

action: "include"

- feature: "Person 1.4_Qualification"

justification: "Known pre-flight"

action: "include"

```
- feature: "Person 1.5_Experience"
  justification: "Known pre-flight, bucket into ordinal bands"
  action: "derive"
# ... additional approved pre-circumstance features
```

Listing A.3: Feature Whitelist Excerpt (configs/feature_whitelist.yaml)

A.4 COST-SENSITIVE PREDICTION ALGORITHM

The cost sensitive prediction stage of the severity triage pipeline computes the predicted class as the class that minimises the expected cost under the configured cost matrix. The expected cost vector for each input is computed as the matrix product of the calibrated probability vector and the cost matrix. The predicted class is the index of the minimum element of the expected cost vector.

python

```
import numpy as np
def build_cost_matrix(cost_config: dict, num_classes: int = 4) -> np.ndarray:
    C = np.zeros((num_classes, num_classes), dtype=float)
    for i in range(num_classes):
        row = cost_config.get(f'actual_{i}', {})
        for j in range(num_classes):
            C[i, j] = row.get(f'pred_{j}', 0.0)
    return C
def predict_cost_sensitive(probs: np.ndarray,
                          cost_config: dict,
                          num_classes: int = 4) -> np.ndarray:
    """
    Choose the prediction that minimises expected cost for each sample.
     $E[\text{cost} \mid \text{pred}=j] = \sum_i P(\text{actual}=i) * C[i, j]$ 
    Parameters
    -----
    probs : np.ndarray, shape (n_samples, num_classes)
        Calibrated class probabilities.
    cost_config : dict
        The costs sub-dict from cost_matrix.yaml.
    Returns
    -----
    np.ndarray of shape (n_samples,) — integer class predictions.
    """
    C = build_cost_matrix(cost_config, num_classes)
    expected_cost = probs @ C      # (n_samples, num_classes)
    return expected_cost.argmax(axis=1)
```

Listing A.4: Cost-Sensitive Prediction (src/models/severity.py)

A.5 PRIOR SHIFT CORRECTION FOR PRE-FLIGHT RISK

The prior shift correction applied to the pre flight risk model recovers deployment ready probabilities from the case control sampled training distribution. The closed form odds adjustment converts the sampled probability to sampled odds, multiplies by the ratio of true prior odds to training prior odds, and converts back to a probability.

python

```
import numpy as np

def adjust_prior(p_sampled, p_true_prior, p_train_prior):
    p_sampled = np.clip(p_sampled, 1e-15, 1 - 1e-15)
    odds_sampled = p_sampled / (1.0 - p_sampled)
    odds_true_prior = p_true_prior / (1.0 - p_true_prior)
    odds_train_prior = p_train_prior / (1.0 - p_train_prior)
    odds_adjusted = odds_sampled * (odds_true_prior / odds_train_prior)
    return odds_adjusted / (1.0 + odds_adjusted)

class PriorShiftedCalibratedModel:
    """
    Wraps a calibrated classifier to apply prior shift correction at
    prediction time. Used when the underlying model was trained on a
    case-control sampled dataset whose positive rate differs from the
    deployment-time true rate.
    """
    def __init__(self, calibrated_model, true_prior, train_prior, features):
        self.calibrated_model = calibrated_model
        self.true_prior = true_prior
        self.train_prior = train_prior
        self.features = features

    def predict_proba(self, X):
        probs = self.calibrated_model.predict_proba(X)
        p_sampled = probs[:, 1]
        p_adjusted = adjust_prior(p_sampled, self.true_prior, self.train_prior)
        return np.vstack([1 - p_adjusted, p_adjusted]).T

    def predict(self, X, threshold=None):
        probs = self.predict_proba(X)[:, 1]
        if threshold is None:
            threshold = self.true_prior * 5
        return (probs >= threshold).astype(int)
```

Listing A.5: Prior Shift Correction (src/models/preflight.py)

A.6 FUSION MULTI-LAYER PERCEPTRON

The Fusion MLP is the third tier of the category classification architecture. It combines Sentence-BERT text embeddings with LightGBM leaf embeddings of tabular features, scales the concatenated input through a StandardScaler, and trains a Multi-Layer Perceptron with binary cross entropy loss and per label sigmoid heads.

python

```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.preprocessing import StandardScaler
class FusionMLP:
    """
```

Two-tower fusion: Sentence-BERT embeddings + tabular features

-> MLP -> Multi-label BCE loss.

"""

```
def __init__(self, hidden_dims=(256, 128), lr=5e-4, max_epochs=30,
              batch_size=128, patience=3, random_state=42):
    self.hidden_dims = hidden_dims
    self.lr = lr
    self.max_epochs = max_epochs
    self.batch_size = batch_size
    self.patience = patience
    self.random_state = random_state
    self.model = None
    self.scaler = None
    self.thresholds = {}
    self.label_names = []

def fit(self, text_train, tabular_train, y_train,
        text_val, tabular_val, y_val, label_names):
    torch.manual_seed(self.random_state)
    self.label_names = label_names
    X_tr_raw = np.hstack([text_train, tabular_train]).astype(np.float32)
    X_va_raw = np.hstack([text_val, tabular_val]).astype(np.float32)
    self.scaler = StandardScaler()
    X_tr = self.scaler.fit_transform(X_tr_raw)
    X_va = self.scaler.transform(X_va_raw)
    Y_tr = y_train[label_names].values.astype(np.float32)
    Y_va = y_val[label_names].values.astype(np.float32)
    input_dim = X_tr.shape[1]
    num_classes = len(label_names)
    self.model = nn.Sequential(
        nn.Linear(input_dim, self.hidden_dims[0]),
        nn.ReLU(), nn.Dropout(0.2),
        nn.Linear(self.hidden_dims[0], self.hidden_dims[1]),
        nn.ReLU(), nn.Dropout(0.2),
        nn.Linear(self.hidden_dims[1], num_classes)
    )
    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.AdamW(self.model.parameters(), lr=self.lr)
    # Training loop with early stopping on validation loss.
    # Per-label thresholds are tuned on validation after best model selection.
    ...
```

Listing A.6: Fusion MLP (src/models/category.py)

A.7 MAIN CONFIGURATION

The main configuration file holds the dataset paths, the global random seed, and the hyperparameter defaults for the principal models. Externalising these values into a versioned configuration file makes the experimental setup reproducible and auditable.

yaml

```

project:
  name: "ASTRA — AI-Driven Safety and Telemetry Risk Analytics"
  version: "1.0.0"
paths:
  raw_data: "data/raw/asrs_full.parquet"
  processed_data: "data/processed/"
  configs: "configs/"
data:
  hf_dataset: "elihoole/asrs-aviation-reports"
  time_col: "Time_Date"
seeds:
  global_seed: 42
model_params:
  # Problem A — LightGBM severity classifier
  lgbm_severity:
    num_leaves: 63
    learning_rate: 0.05
    n_estimators: 1000
    early_stopping_rounds: 50
    min_child_samples: 30
    subsample: 0.8
    colsample_bytree: 0.8
    class_weight: "balanced"
  # Problem A — CatBoost severity classifier (deployed model)
  catboost_severity:
    iterations: 800
    learning_rate: 0.05
    depth: 6
    early_stopping_rounds: 50
    class_weights_mode: "Balanced"
  # Problem D — BERTopic clustering
  bertopic:
    min_topic_size: 50
    nr_topics: 30
    pca_components: 15
    kmeans_batch_size: 1024
  # Problem D — PELT changepoint detection
  pelt:
    penalty: 3.0
    min_signal_length: 10
  # Problem E — Heterogeneous co-occurrence graph
  graph:
    min_edge_weight: 5
    min_severity_threshold: 1.5
    top_k_centrality: 15
    top_k_paths: 20

```

Listing A.7: Main Configuration (configs/main_config.yaml)